PDF 필기 자료 (코넬 노트)

2026년 1학기 · Games, CSP/Factor Graph, Markov/Bayesian Net, Logic · 총 35쪽

인공지능 필기본 — 코넬

Games · CSP/Factor Graph · Markov/Bayesian Net · Logic (강의 09-18) · 회독용

Ch 09 Games 1 — Minimax · Evaluation · Alpha-Beta

2-player zero-sum game tree search · depth-limited eval · alpha-beta pruning

Game Tree & Minimax p.5-20

Q. Minimax V_max/V_min 점화식은?

Q. 상대를 어떻게 가정하나?

Q. terminal·utility의 정의는?

Two-player zero-sum game — 정의 — agent 효용이 올라가면 opp 효용이 같은 크기로 내려가는 게임.

체스 예: Utility= +∞ 백 승 / 0 무승부 / −∞ 흑 승

- **s_start** : 시작 상태
- `Actions(s)` : s에서 가능한 행동 집합
- `Succ(s,a)` : (s,a) → 다음 상태
- `IsEnd(s)` : 종료 여부
- `Utility(s)` : 종료 상태에서만 정의되는 agent 효용
- `Player(s)` ∈ {agent, opp} : s를 제어하는 플레이어

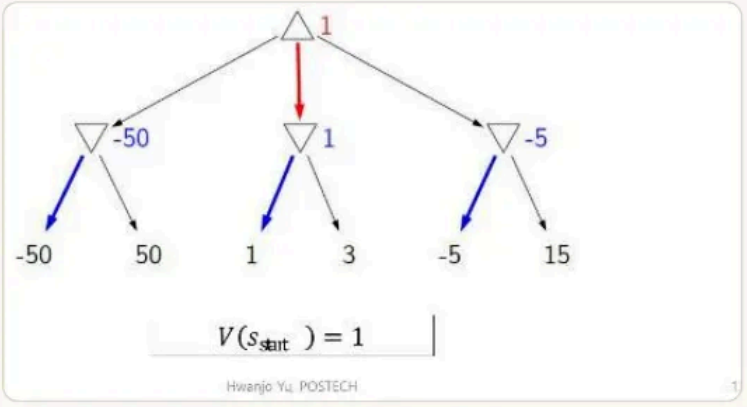
Game Tree — 핵심 아이디어 — 각 노드 = 플레이어의 *결정점*. root-to-leaf 경로 = 게임 가능 결과 하나.

- △ max-node(agent 차례) : 효용 최대화
- ▽ min-node(opp 차례) : 효용 최소화
- ◇ chance-node : 확률 가중 기댓값

Minimax 점화식 — opp가 최악(agent에게 불리)을 선택한다고 가정. MDPs에 대응 없음.

opp 최악 가정 = minimax

- IsEnd(s): V(s) = Utility(s)
- Player(s)=agent: V(s) = max over a∈Actions(s) of V(Succ(s,a))
- Player(s)=opp: V(s) = min over a∈Actions(s) of V(Succ(s,a))



Minimax example — V(s_start)=1 (p.15)

정책 추출

- π_agent(s) = argmax_a V(Succ(s,a))
- π_opp(s) = argmin_a V(Succ(s,a))

Minimax 두 성질

- **Property 1 — Best against minimax opponent**: V(π_agent, π_opp)(s) ≥ V(π, π_opp)(s) for all π → minimax 상대에게는 minimax 정책이 최선
- **Property 2 — Lower bound against any opponent**: V(π_agent, π_opp)(s) ≤ V(π_agent, π)(s) for all π → 어떤 상대든 minimax 정책이면 V(π_agent,π_opp)가 하한 보장
- **Non-property 3**: minimax 정책이 모든 상대에게 최적은 아님 — opp가 최악이 아닌 경우 손해 가능

✎ 핵심 minimax = 최악 상대 가정 → 보수적 보장. opp가 실수하면 기대보다 더 잘함. 'non-property 3'이 핵심 — 무조건 최적이 아님.

Expectiminimax — 확장 — Players에 coin(chance) 추가 시 점화식에 chance-node 추가.

- Player(s)=coin: V(s) = Σ_a p(s,a)·V(Succ(s,a)) (확률 가중합)
- Primitives: max, min, chance-node를 *모델에 따라 조합*하면 어떤 게임도 표현 가능

Minimax 복잡도 — 분기 인수 b, 깊이 d(ply 단위). 공간 O(bd), 시간 O(b^d).

- 체스: b≈35, d≈50 → 완전 탐색 불가(천문학적)
- 두 가지 가속: (1) Evaluation function — 근사값, (2) Alpha-beta pruning — 정확값

Evaluation Functions p.27-36

Q. depth-limited search 가 왜 필요?

Q. evaluation function 의 역할은?

Q. 근사일 뿐인 이유는?

Depth-limited Search — 최대 깊이 d_max에서 탐색 멈추고 Eval(s) 호출. 시간: O(b^d_max)

- V(s, d) = Eval(s) if d=0 or IsEnd(s)
- max/min over successors of V(Succ, d−1) otherwise
- 관례: 마지막 플레이어 차례에 depth 1 차감

Eval(s) — 정의 & 성질 — V(s)의 (약한) *추정치*. 검색 문제의 FutureCost(s)에 대응.

- 체스 예: Eval(s) = material + mobility + king-safety + center-control
- material = 10(K−K') + 9(Q−Q') + 5(R−R') + 3(B−B') + 3(N−N') + 1(P−P')

선형 함수 근사 — Parameterized Eval — Eval(s; w) = w^T · f(s). w∈ℝⁿ 가중치, f(s) = 특징 벡터.

- f_i = (내 말 개수) − (상대 말 개수) 등 도메인 특징
- w를 학습으로 결정 → supervised / self-play 강화학습

참 가치함수 근사 — 두 방법 — 최적 정책 π*, π'_opp를 알면 Eval(s)=V(s) (intractable). 두 근사:

- ① Stochastic policies: π, π'를 확률 정책으로 교체 → 단순화되나 여전히 계산량 있음
- ② Monte Carlo: K번 random rollout → Eval(s) ≈ (1/K)Σ util_k

💡 TIP Monte Carlo Eval: K=10 예시 → (1+3+50+50+50−50−50+50+15−5)/10 = 11.4. AlphaGo = MCTS(탐색) + policy net(스토캐스틱 π) + value net(Eval).

Alpha-Beta Pruning p.38-44

Q. α·β의 의미는?

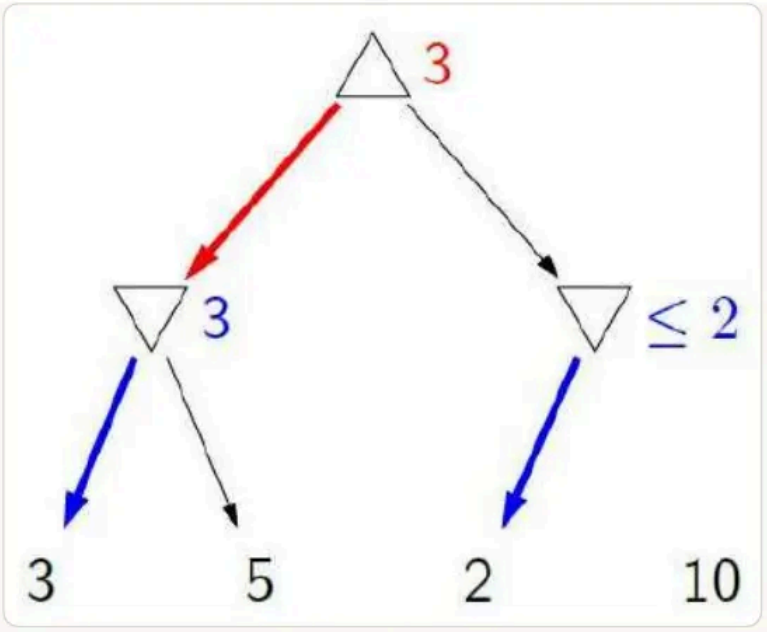
Q. 가지치기 조건(β≤α)은?

Q. 최선 정렬 시 복잡도는?

Q. 답이 보존되는 이유는?

프루닝 원리 — Branch and Bound — 구간 [α, β]가 겹치지 않으면 추가 계산 없이 선택 가능.

- 예: A=[3,5], B=[5,100] → max 선택은 B로 확정, A 추가 탐색 불필요
- Alpha-beta = BnB를 minimax 트리에 특화한 것



β≤2이므로 우측 서브트리 불필요 — max(3,≤2)=3 (p.39)

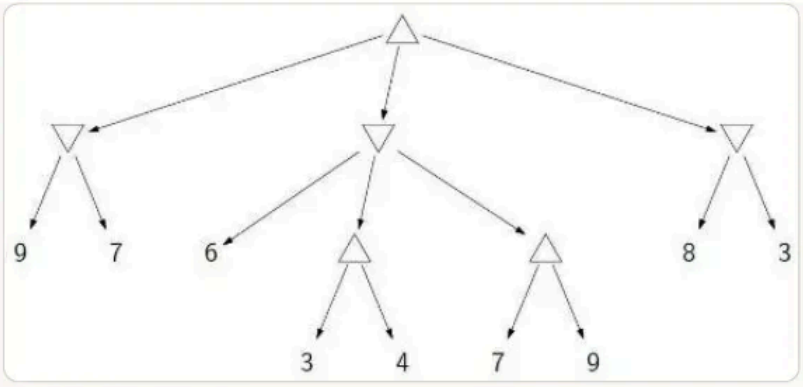
α·β 의미

- α = max-node의 하한 — 지금까지 본 자식 중 최댓값
- β = min-node의 상한 — 지금까지 본 자식 중 최솟값
- Optimal path의 모든 노드 값은 동일 (optimal path property)
- 조상 중 max-node의 α 최댓값 = A, min-node의 β 최솟값 = B로 유지
- Prune 조건: β ≤ α (상한이 하한 이하 → 구간 교집합 없음)

Alpha-Beta Pruning

```
function alphabeta(node, depth, α, β, maximizingPlayer)
  if depth = 0 or node is terminal
    return Eval(node)
  if maximizingPlayer
    v := −∞
    for each child of node
      v := max(v, alphabeta(child, depth−1, α, β, FALSE))
      α := max(α, v)
      if β ≤ α   ← β-cutoff: min-ancestor가 이미 β≤α 확인
        break
    return v
  else
    v := +∞
    for each child of node
      v := min(v, alphabeta(child, depth−1, α, β, TRUE))
      β := min(β, v)
      if β ≤ α   ← α-cutoff: max-ancestor가 이미 α≥β 확인
        break
    return v
호출: alphabeta(root, depth, −∞, +∞, TRUE)
```

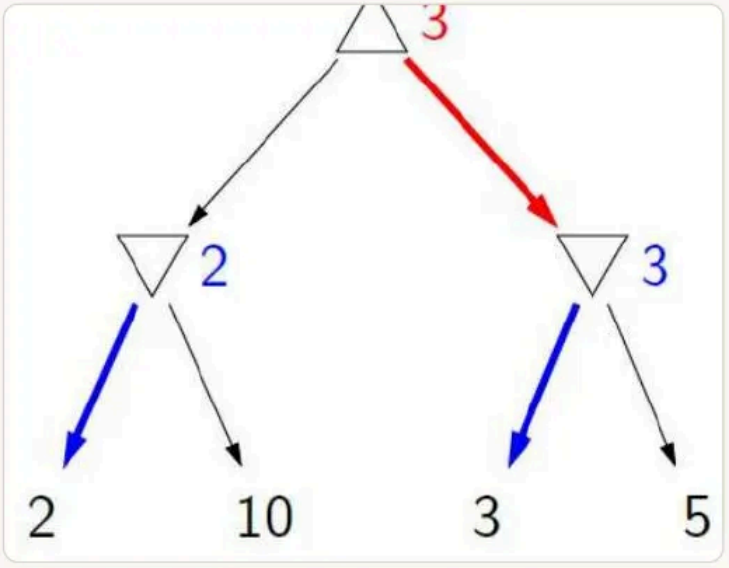
β≤α = 상한이 하한 이하 → 이 서브트리가 조상 선택에 영향 못 줌 → prune.



Alpha-beta pruning example tree (p.41)

Move Ordering — 복잡도 — Pruning 효과는 행동 탐색 순서에 의존.

- 최악 순서: O(b^d) — pruning 없음
- 최선 순서: O(b^(d/2)) — 홀수 깊이마다 1개만 탐색, effective branching ≈ √b
- 랜덤 순서: O(b^(3d/4))
- 실전: `Eval(s)` 로 max-node는 내림차순, min-node는 오름차순 정렬



순서 바뀌면 5-node pruning 불가 (p.43)

	Worst ordering	Best ordering	Random ordering
시간복잡도	O(b^d)	O(b^(d/2))	O(b^(3d/4))
chess 효과	b=35,d=50 불가	b=6,d=50 가능권	중간

move ordering 품질이 pruning 효율을 O(b^d/2) vs O(b^d) 차이만큼 바꿈

★ 필수암기 α-cutoff: β≤α → max-node에서 break. β-cutoff: β≤α → min-node에서 break. 최선 정렬 시 effective 분기 √b → 같은 시간에 2배 깊이 탐색. minimax 값 변경 없이 동일 결과(exact, not approximate).

⚠ 함정 Alpha-beta pruning은 정확한 답을 내지만 평균 pruning 비율은 move ordering에 의존. Eval 호출 비용 vs pruning 이득 trade-off 고려.

요약 Minimax = max/min 점화식으로 **최악 상대 가정** 하 V(s) 계산 · Eval+depth-limit으로 근사 · Alpha-Beta는 β≤α 시 서브트리 prune(답 불변) · 최선 정렬 시 **O(b^(d/2))**로 깊이 2배 탐색.

Ch 10 Games 2 — TD Learning · Simultaneous · Non-zero-sum

강화학습 기반 게임 평가함수 학습 + 동시수행 게임 minimax + Nash 균형

TD Learning p.1-15

Q. TD error(prediction–target)는?

Q. w 갱신식은? on-policy인 이유?

Q. Q-learning이 off-policy인 까닭?

평가함수 파라미터화 — 보드 상태 `s` 를 feature vector `ϕ(s)` 로 표현, **가중치 w** 를 학습. `V(s;w) = w·ϕ(s)` (선형 함수일 때).

- Backgammon 예: # of pieces in each column, fraction removed 등 **수치 feature** 사용
- Self-play로 데이터 생성 → π_agent(max), π_opp(min) 기반 episode

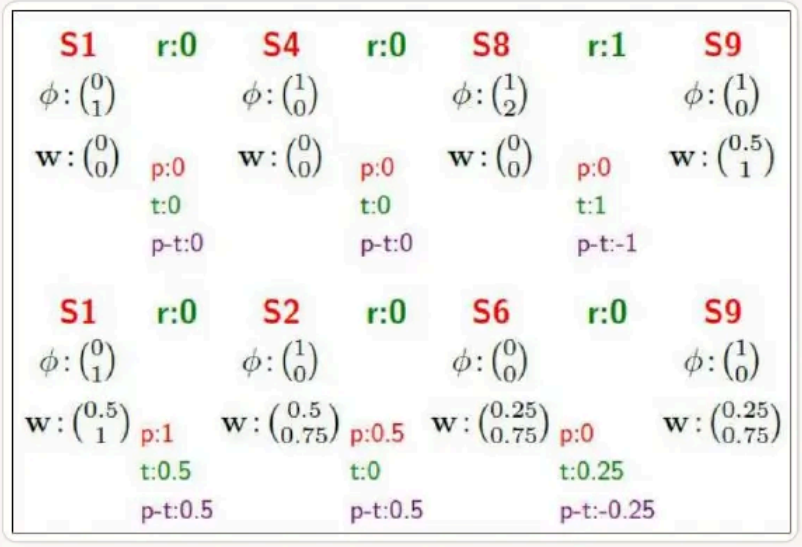
Episode & Prediction-Target 구조 — 에피소드: $s_0;\ a_1,r_1,s_1;\ a_2,r_2,s_2;\ \ldots;\ a_n,r_n,s_n$

- Prediction = `V(s;w)` — 현재 가중치의 예측
- Target = `r + γ·V(s´;w)` — 다음 상태 + 즉시보상으로 만든 목표치
- 목적함수: ½(prediction − target)² 를 최소화

TD Learning (on-policy)

```
On each transition (s, a, r, s'):
  w ← w − η · [V_π(s;w) − (r + γ·V_π(s';w))] · ∇_w V_π(s;w)
For linear functions:
  V(s;w)      = w · ϕ(s)      ← 선형 평가함수
  ∇_w V(s;w) = ϕ(s)           ← gradient = feature vector
```

η=학습률, γ=discount. prediction–target = TD error. ∇=feature vector(선형).



TD learning 추적 예제 — η=0.5, γ=1, 종단 보상만 (p.9)

Q-Learning (off-policy TD)

```
On each transition (s, a, r, s'):
  Q(s,a;w) ← Q(s,a;w) − η · [Q(s,a;w) − (r + γ·max_{a'} Q(s',a';w))] · ∇Q(s,a;w)
```

off-policy: 다음값이 optimal policy 기반(max). MDP transition 불필요.

TD vs Q-Learning 비교

- TD learning: `V(s;w)` 연산 · on-policy(탐색 정책 기반) · Succ(s,a) 필요
- Q-learning: `Q(s,a;w)` 연산 · off-policy(최적 정책 추정) · transition model 불필요

✎ **핵심** TD error = prediction − target. 선형일 때 gradient = `ϕ(s)` 자체. Q-learning은 max a' 를 써서 off-policy, TD는 π 기반이라 on-policy.

Simultaneous Games p.16-28

Q. pure vs mixed strategy 차이?

Q. minimax theorem(max min=min max)?

Q. 동시게임 payoff matrix 읽기?

Single-move Simultaneous Game 정의 — Players {A,B}, 각자 동시에 action 선택. `V(a,b)` = A의 utility (B의 utility = −V, zero-sum).

- **Payoff matrix** V: 행=A의 action, 열=B의 action
- Two-finger Morra 예: V(1,1)=+2, V(1,2)=−3, V(2,1)=−3, V(2,2)=+4

A \ B	1 finger	2 fingers
1 finger	+2	−3
2 fingers	−3	+4

Two-finger Morra payoff matrix — A의 관점(양수=A 이득)

Strategy 정의

- **Pure strategy** — 단일 action `a ∈ Actions` (결정론적)
- **Mixed strategy** — 확률분포 `π` s.t. $0 \le \pi(a) \le 1$, $\Sigma\pi(a)=1$ (확률론적)
- Game value: $V(\pi_A, \pi_B) = \Sigma_{a,b}\ \pi_A(a)\cdot\pi_B(b)\cdot V(a,b)$

Pure strategy — 선공의 불이익 — A 선공: $\max_a \min_b V(a,b)$ ≤ B 선공: $\min_b \max_a V(a,b)$

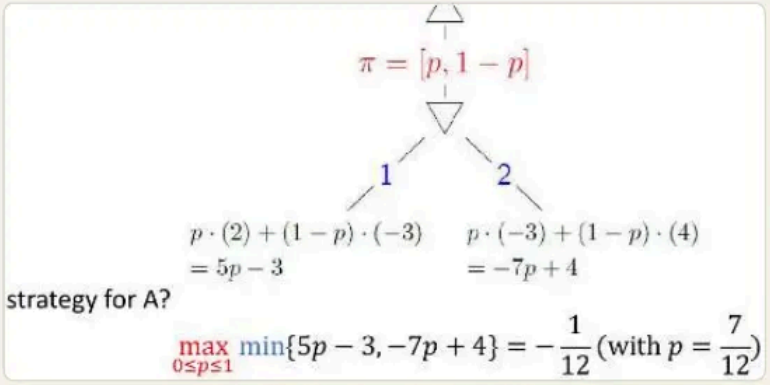
- 명제: **나중에 두는 것이 불리하지 않다** — 상대 전략을 보고 최적 대응 가능
- 이 부등식은 pure strategy에서 일반적으로 *등호가 성립하지 않는다*

Mixed strategy — second player는 pure로 충분 — 임의의 고정 mixed π_A 에 대해, $\min_{\pi_B} V(\pi_A,\pi_B)$ 는 pure strategy π_B 로 달성 가능.

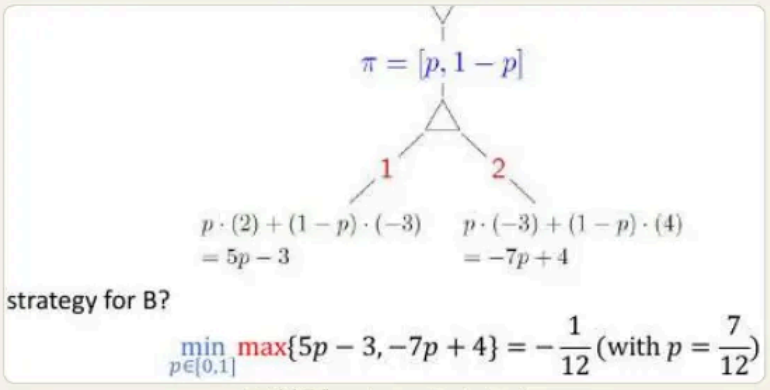
- 이유: $V(\pi_A, \pi_B)$는 π_B에 대해 **선형(linear/convex combination)** → 최솟값은 꼭짓점(pure)

Mixed strategy 최적 계산 — 과정 — Morra 예: A가 먼저 $\pi=[p,1-p]$ 를 공개. B는 pure 1 또는 2 선택.

- B가 1 선택 시 value = $p\cdot 2+(1-p)\cdot(-3) = 5p-3$
- B가 2 선택 시 value = $p\cdot(-3)+(1-p)\cdot 4 = -7p+4$



① A 먼저 공개 — max_p min{5p−3, −7p+4}



② B 먼저 공개 — min_p max{5p−3, −7p+4}

Mixed strategy game tree — A/B 순서 무관, 최적값 동일 (p.25-26)

Minimax 정리 (von Neumann 1928) — 유한 행동 2인 제로섬 게임에서 mixed strategy 범위로: max_π_A min_π_B V = min_π_B max_π_A V

- 최적 mixed strategy를 먼저 공개해도 손해 없음 — **전략 노출이 불이익이 되지 않는다**
- 두 순서 모두 동일한 game value (Morra: `−1/12`, p=7/12에서)
- 두 순서의 부등식이 등호로 성립: pure에서는 ≤, mixed에서는 =

> ★ **필수암기** minimax theorem: mixed strategy 시 max min = min max (등호). Pure strategy에서는 max min ≤ min max(부등호). Second player는 always pure strategy로 최적 달성 가능.

Non-zero-sum Games & Nash Equilibrium p.29-34

Q. Nash equilibrium 정의(단독이탈)?

Q. 존재정리 — 항상 존재?

Q. Pareto 최적과의 관계 (죄수딜레마)?

Utility 구조에 따른 게임 분류

- **Competitive (zero-sum)**: minimax로 풂 (linear programming)
- **Collaborative**: pure maximization (plain search)
- **Non-zero-sum**: 각 플레이어 utility `U_i(a,b)` 별도 — minimax 불성립

Prisoner's Dilemma — 대표 Non-zero-sum — A와 B 각각 testify/refuse 선택. 서로 다른 utility 구조.

- (testify, testify): U_A=−5, U_B=−5
- (testify, refuse): U_A=0, U_B=−10
- (refuse, testify): U_A=−10, U_B=0
- (refuse, refuse): U_A=−1, U_B=−1

A \ B	testify	refuse
testify	−5, −5	0, −10
refuse	−10, 0	−1, −1

Prisoner's Dilemma payoff (U_A, U_B) — Nash equilibrium = (testify, testify)

Nash Equilibrium 정의 — (π*_A, π*_B)가 Nash equlibrium ↔ 어느 플레이어도 단독으로 전략을 바꿀 인센티브가 없는 안정점

- `U_A(π*_A, π*_B) ≥ U_A(π_A, π*_B)` for all π_A
- `U_B(π*_A, π*_B) ≥ U_B(π*_A, π_B)` for all π_B
- Nash's existence theorem: 유한 플레이어·유한 행동 게임에서 Nash equilibrium이 반드시 존재한다

Nash Equilibrium 예시

- Two-finger Morra (zero-sum): A·B 모두 `π=[7/12, 5/12]` — minimax 해와 일치
- Collaborative Morra: Nash equilibrium 2개 — (둘다 1, value=2) / (둘다 2, value=4)
- Prisoner's Dilemma: (testify, testify) — 개인 이성이 집단 최적(refuse,refuse)을 달성 못 함
- Collaborative game: Nash eq = 행·열 내 다른 셀이 모두 더 작지 않은 항

⚠ **함정** Prisoner's dilemma: (refuse,refuse)이 Pareto 최적이나 Nash equilibrium이 아님. A가 단독으로 testify로 바꾸면 −10→0으로 이득. Nash eq = (testify,testify) — 개인합리성이 집단비효율을 낳는 전형적 트랩.

Zero-sum vs Non-zero-sum 비교

- Zero-sum: minimax theorem — unique game value, multiple minimax strategies
- Non-zero-sum: Nash theorem — multiple Nash equilibria, multiple game values 가능

	Zero-sum	Non-zero-sum
이론	von Neumann minimax (1928)	Nash existence (1950)
해법	Linear programming	Nash equilibrium 탐색
game value	유일(unique)	여러 개 가능
전략 공개	최적 mixed 공개 무해	공개 전략이 불이익될 수 있음

동시수행 게임 — zero-sum vs non-zero-sum 핵심 비교

★ **필수암기** 정리: TD learning w ← w−η[V(s)−(r+γV(s'))]∇V(s). Mixed minimax = max min = min max (pure에선 ≤). Nash eq = 누구도 단독이탈 불가인 안정점. Prisoner's dilemma Nash eq = (testify,testify), Pareto opt = (refuse,refuse).

요약 TD learning: **prediction−target=TD error**로 w 갱신(on-policy) · Q-learning은 max a'로 off-policy · zero-sum 동시게임은 **mixed strategy → minimax theorem(max min=min max)** · non-zero-sum은 **Nash equilibrium** 항상 존재(단 Pareto 최적은 미보장, 죄수의 딜레마).

Ch 11 Factor Graphs & CSP 1

변수-기반 모델 · Factor Graph 정의 · CSP 모델링

State-based vs Variable-based Models p.3-10

Q. state-based vs variable-based 차이?

Q. 변수기반이 유리한 문제 유형은?

State-based models 한계 — 순서(order)가 중요한 탐색·MDP는 state가 핵심. 그러나 순서가 무관한 문제에서는 비효율.

- takeaway 1: 지역 상호작용(local interactions)을 지정 → 전역 최적(global optimum) 탐색
- takeaway 2: state = 미래 최적 행동 선택에 충분한 과거 요약
- 순서가 무관(irrelevant)한 문제 → variable-based models가 더 적합

Variable-based models — 변수들에 값을 할당하는 *assignment* 문제. 대표: CSP, PGM(Markov/Bayesian networks, HMM, CRF).

- Modeling: 변수·도메인·팩터(제약) 지정
- Algorithm: 할당 탐색(exact / approximate)

✎ **핵심** 핵심 구분: state-based(순서 중요·MDP·탐색) vs variable-based(순서 무관·CSP·PGM). CSP는 variable-based의 all-or-nothing(0/1) 팩터 특수 사례.

Factor Graph p.13-19

Q. 변수·팩터 노드의 의미는?

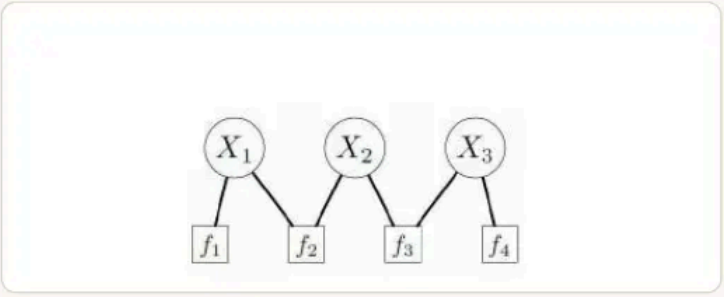
Q. Weight(x)=∏f ⱼ(x) 정의는?

Q. assignment의 weight 계산법?

Factor graph 정의

변수=원 팩터=사각형

- **Variables**: X = (X₁, ⋯, Xₙ), X_i ∈ Domain_i
- **Factors**: f₁, ⋯, fₘ, 각 f_j(X) ≥ 0 (비음수)
- 그래프 표현: 원(variable) + 사각형(factor) + 간선(scope 연결)



Factor graph 구조 — X₁,X₂,X₃와 f₁~f₄ (p.14)

Scope & Arity — factor f의 scope = 그 팩터가 의존하는 변수 집합. arity = scope의 크기.

- Unary factor: arity 1 (변수 1개에만 의존)
- Binary factor: arity 2 (변수 2개에 의존)
- **Constraint**: f ∈ {0,1} 인 팩터. 1=만족, 0=위반

Assignment Weight — 각 assignment x = (x₁,⋯,xₙ)의 weight = 모든 팩터값의 곱.

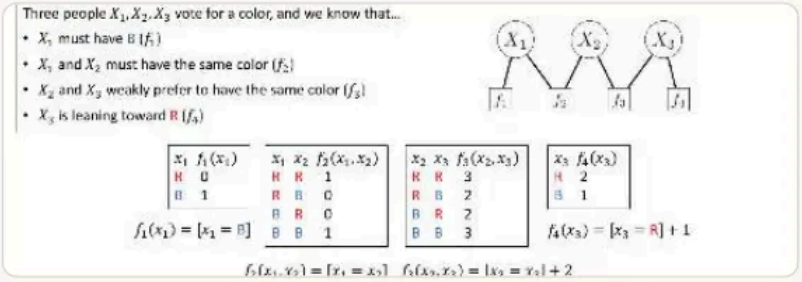
veto power: 하나라도 0이면 weight=0

- Weight(x) = ∏ⱼ f ⱼ(x) ← 곱, 합이 아님
- 어느 한 팩터라도 0이면 전체 weight = 0 (veto power)
- **목적**: arg max_x Weight(x) 를 찾는 것

> ⚠ **함정** Weight는 곱(product)이다. 더하기(sum)로 혼동 금지. Constraint factor가 하나라도 위반되면 전체 weight=0 → 자동 제거.

예제: 투표 (voting) — X₁,X₂,X₃ ∈ {R,B}. 4개 팩터(단항 2개·이항 2개). Weight 계산.

- f₁(x₁)=[x₁=B], f₂(x₁,x₂)=[x₁=x₂], f₃(x₂,x₃)=[x₂=x₃]+2, f₄(x₃)=[x₃=R]+1
- 최대 weight 할당: (B,B,R) ⟶ Weight = 1·1·2·2 = 4



투표 예제 — 팩터 테이블과 전체 weight 계산 (p.13)

Constraint Satisfaction Problem (CSP) p.20-21

Q. CSP가 factor graph의 특수사례인 이유?

Q. consistent assignment 조건은?

Q. constraint의 veto power란?

CSP 정의 — 모든 팩터가 constraint(0 or 1)인 factor graph 특수 사례.

- f ⱼ(x) ∈ {0,1} for all j
- **Consistent assignment**: Weight(x) = 1, 즉 모든 constraint 만족
- **목적**: consistent assignment 존재 여부 판별 (또는 탐색)

	Factor graph (일반)	CSP (특수)
단위	변수(variable)	변수(variable)
평가 함수	팩터 f_j(x) ≥ 0	제약 f_j(x) ∈ {0,1}
목적	max Weight(x)	consistent assignment 탐색
weight=0 의미	나쁜 할당	제약 위반 (불가)

Factor graph vs CSP 비교

CSP 모델링 p.22-28

Q. 변수·도메인·제약 설계 절차?

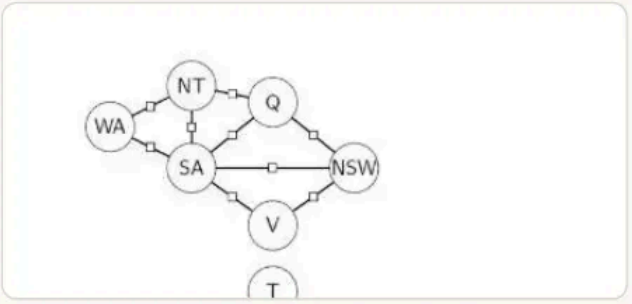
Q. 좋은 모델링의 기준은?

모델링 절차 — 문제 → CSP로 변환하는 4단계.

- ① **변수·도메인 결정**: 무엇을 모를지 변수로, 도메인은 가능한 값 집합
- ② **팩터(제약) 설계**: 각 요구사항을 constraint로 변환
- ③ **크기 최소화**: 변수·팩터·도메인·arity를 작게 유지
- ④ **검증 관점**: 해를 구하는 게 아니라 주어진 해가 맞는지 체크하는 방향으로 팩터 구현

예제: Map Coloring (호주 7개 주) — 7개 주를 {R,G,B}로 칠하되, 인접 주는 다른 색.

- Variables: X = (WA, NT, SA, Q, NSW, V, T), Domain_i = {R,G,B}
- Factors(constraints): [WA≠NT], [NT≠Q], [NT≠SA], [SA≠Q], [SA≠NSW], [SA≠V], [Q≠NSW], [NSW≠V]
- Weight([각 인접쌍 다른색]) = 1 ← consistent
- Tasmania(T)는 **독립(isolated)** → 다른 변수에 영향 없음



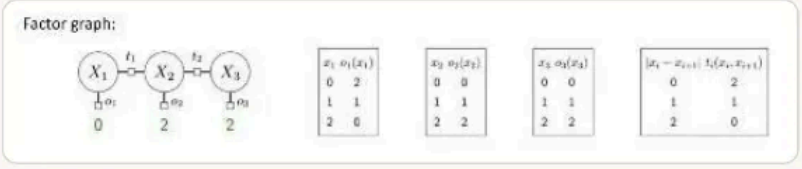
Map coloring constraint graph (p.15)

예제: LSAT (Art Gallery) — 조각품 A,B,C → 방 1,2에 배치. 3개 조건.

- C1: A,B 같은 방 불가 [room(A)≠room(B)]
- C2: B,C 같은 방 [room(B)=room(C)]
- C3: 방 2는 하나만 [Σ{i:room(i)=2} 1 ≤ 1]
- C1∧C2: A≠B, B=C ⟹ A≠C. C3: 방2에 1개만 ⟹ 해: A→방2, B,C→방1 (또는 A→방1, B,C→방2이면 C3 위반)

예제: Object Tracking — 센서 관측 0,2,2 → 물체 실제 경로 추정. CSP + 연속 시간 구조.

- Variables: X_i ∈ {0,1,2} — 시각 i의 위치
- Observation factors o_i(x_i): 센서값과의 호환성 (노이즈 허용)
- Transition factors t_i(x_i,x_{i+1}): |x_i − x_{i+1}| ≤ 1 (텔레포트 금지)
- Factor graph = 체인(chain) 구조 → **순서 있는 변수** 간 국소 제약



Object tracking factor graph — 체인 구조 + 팩터 테이블 (p.23)

예제: Event Scheduling (두 가지 정식화) — E개 이벤트, T개 시간슬롯. C1/C2/C3 만족.

- Formulation 1 — 변수: X_e ∈ {1,⋯,T} (이벤트→슬롯). C2: 모든 e≠e′에 대해 [X_e≠X_{e′}] → 팩터 수 $O(E^2)$. C3: [(e,X_e)∈A]
- Formulation 2 — 변수: Y_t ∈ {1,⋯,E}∪{∅} (슬롯→이벤트). C1: 각 e가 정확히 하나의 t에 배정. C3: [(Y_t=∅) or (Y_t,t)∈A]
- 같은 문제라도 변수 정의 방향에 따라 팩터 구조·수가 달라짐

예제: Program Verification — def foo(x,y): a=x²; b=a+y²; c=b-2xy; return c. Spec: c≥0 for all x,y.

- Variables: x, y, a, b, c
- Constraints (프로그램 구문): [a=x²], [b=a+y²], [c=b-2xy]
- Constraint (명세 부정): [c<0]
- CSP에 consistent assignment 없으면 → 프로그램이 명세 만족 (반례 없음)
- CSP에 consistent assignment 있으면 → 반례 존재 (버그)

✎ 핵심 Program verification의 CSP 핵심: 명세의 부정(negation)을 constraint로 추가. CSP UNSAT ⟺ 프로그램 정확. 프로그램 assignment(=) → CSP equality(=)로 변환.

요약 & 필수암기 p.28

Q. factor graph ↔ CSP 핵심 한 줄?

CSP 모델링 체크리스트

- 변수·도메인 결정 → 팩터(constraint) 설계 → 크기 최소화 → 검증 관점으로 구현
- Factor graph Weight = $\prod f_j(x)$ (곱), CSP는 0/1 팩터만
- Consistent ⟺ Weight=1 ⟺ 모든 constraint 만족
- 같은 문제에 formulation이 여러 개 가능 → 팩터 수·복잡도 다름

★ 필수암기 Factor graph: Variables X_i∈Domain_i, Factors f_j≥0, Weight(x)=∏f_j(x), 목적=arg max_x Weight(x). CSP: 팩터 all 0/1, consistent=Weight=1. Scope=factor가 의존하는 변수집합, Arity=scope크기. Veto power: 한 팩터=0이면 전체 weight=0.

요약 Factor graph = 변수(원)+팩터(사각형) 이분 그래프, Weight(x)=$\prod f_j(x)$ 최대화 · CSP = 팩터가 모두 0/1인 특수 사례(consistent=Weight 1) · 모델링 핵심=변수·제약 설계 + veto power.

Ch 12 Factor Graphs & CSP 2

Backtracking · Arc Consistency · Beam Search · Local Search (ICM)

Factor Graph & Assignment 기본 정의 p.3-6

Q. partial vs complete assignment?

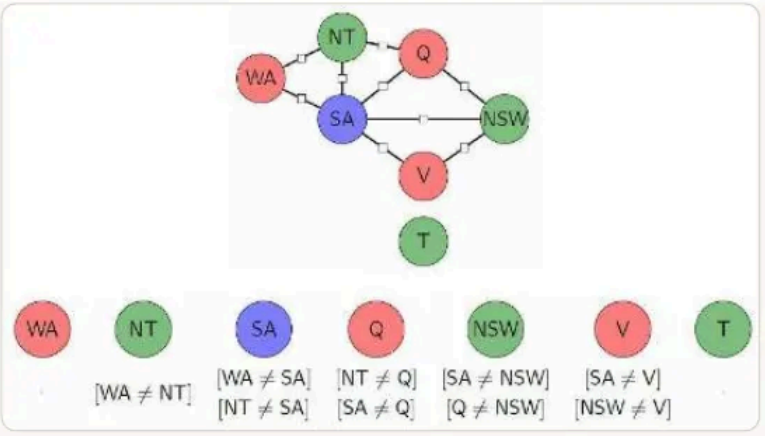
Q. weight·consistency 정의는?

Factor graph — 변수 $X=(X_1,\cdots,X_n)$ + factor $f_1,\cdots,f_m$ (각 $f_j\geq 0$). 목표: max Weight(x) = $\max_x \prod_j f_j(x_scope(j))$

- Assignment weight = $\prod_j f_j(x)$ — 모든 factor의 곱
- Partial assignment a: 일부 변수에만 값 배정. weight = *scope가 배정된 변수만인 factor들의 곱*

Dependent factors — 변수 X_i 배정 시 활성화되는 factor 집합. $dep(a, X_i) = \{ f_j : scope(f_j) \subseteq assigned(a) \cup \{X_i\} \}$

- 예: a={WA:R, NT:G}, X_i=SA → dep = {[WA≠SA], [NT≠SA]}
- 배정 직후 즉시 계산 가능한 partial weight 인수들



Partial assignment weights & dependent factors (p.5)

Backtracking Search p.7-13

Q. MCV(most constrained var) 휴리스틱?

Q. LCV(least constrained value)?

Q. exact·복잡도 특성은?

Backtracking 기본 아이디어 — DFS로 변수에 값을 하나씩 배정. weight=0 가지치기: partial weight가 0이면 그 서브트리 전체 포기. 완전 배정에 도달하면 best 갱신.

- 시간복잡도: $O(|Domain|^n)$ — 모든 조합 탐색

Backtrack(a, w_best, Domains)

```
if a is complete assignment:
  update best(w_best); return
Choose unassigned VARIABLE X_i     ← MCV heuristic
Order VALUES Domain(X_i)           ← LCV heuristic
for each value v in that order:
  w ← ∏_{f∈dep(a,X_i)} f(a ∪ {X_i:v})
  if w = 0: continue               ← prune
  Domains' ← Domains via LOOKAHEAD ← forward checking / AC-3
  if any Domains'[X] = ∅: continue ← domain wipeout
  Backtrack(a ∪ {X_i:v}, w_best, Domains')
```

w=0 가지치기가 핵심. Lookahead로 domain wipeout 조기 감지.

동적 순서 결정 휴리스틱 — 어떤 변수·값을 먼저 배정하느냐로 탐색 공간을 크게 줄일 수 있다.

MCV=fail fast
LCV=solution 유리

- MCV (Most Constrained Variable) — domain 크기 최소 변수 먼저. 어차피 모든 변수를 배정해야 하므로 *fail early*: 문제 있으면 빨리 감지
- LCV (Least Constrained Value) — 이웃 변수에 남은 일관값 수가 최대인 값 먼저. 해에 도달할 가능성 최대화

✎ 핵심 MCV는 constraint(0/1 factor)가 있을 때 유효. LCV는 모든 factor가 constraint(weight∈{0,1})일 때만 유효. Forward checking이 없으면 domain 정보가 갱신되지 않아 둘 다 무력화됨.

Lookahead: Forward Checking & AC-3 p.8-21

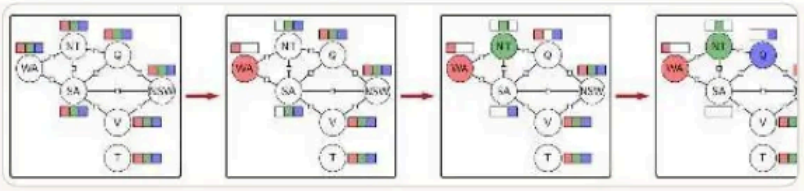
Q. forward checking 동작은?

Q. AC-3 arc consistency 절차?

Q. arc-consistent여도 못 잡는 것은?

Forward Checking (1-step lookahead) — $X_i = v$ 배정 직후, X_i의 모든 이웃 X_j에 대해 비일관 값(x_j s.t. $f(X_i=v, X_j=x_j)=0$)을 Domain(X_j)에서 즉시 제거.

- Domain(X_j)가 비면(empty) → *즉시 return*: 이 브랜치 불가
- 한 단계 이웃만 검사 → 빠르지만 전파 범위 제한적



Forward checking 단계별 domain 갱신 (p.8)

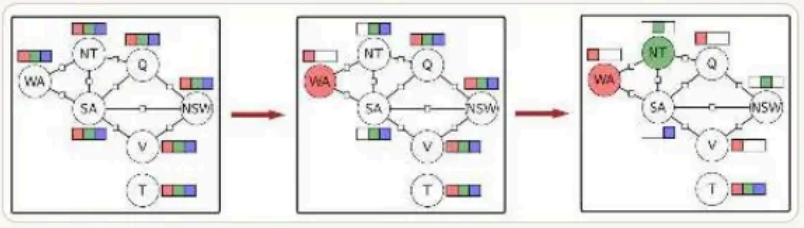
Arc Consistency 정의 — 변수 X_i가 X_j에 대해 **arc-consistent** ↔ $\forall x_i \in \text{Domain}(X_i)$, $\exists x_j \in \text{Domain}(X_j)$: 모든 X_i·X_j 공유 factor에서 $f(X_i=x_i, X_j=x_j)\neq 0$

- 예: $X_i \in \{1,2,3,4,5\}$, $X_j \in \{1,2\}$, factor=$[X_i+X_j=4]$ → X_i를 {2,3}으로 축소

AC-3

```
Queue Q ← {(X_i, X_j) : 모든 arc}
while Q is non-empty:
  Remove any (X_i, X_j) from Q
  EnforceArcConsistency(X_i, X_j)   ← X_i를 X_j 기준으로 축소
  if Domain(X_i) changed:
    for each neighbor X_k of X_i (k≠j):
      add (X_k, X_i) to Q           ← 변화 전파
```

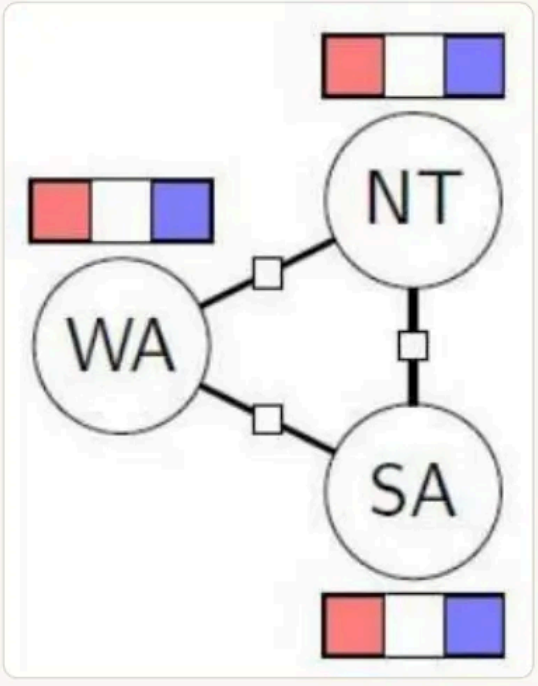
Forward checking = AC-3를 배정된 변수의 이웃에만 1회 실행. AC-3 = 전파 반복.



AC-3 domain 전파 예제: 3단계 (p.18)

AC-3 한계 — **지역적 일관성만 보장**: 쌍(pair)마다 arc-consistent해도 전역(global) 해가 없을 수 있음.

- 예: WA·NT·SA가 모두 arc-consistent지만 3-coloring 불가
- 이유: 삼각형 같은 구조는 두 변수씩 볼 때 문제가 안 보임



AC-3로 검출 불가한 불일치 예 (p.20)

⚠ **함정** AC-3 후 domain이 모두 비어있지 않아도 **해가 없을 수 있다**. Arc consistency ≠ global consistency.

	Forward Checking	AC-3
범위	배정 변수의 이웃 1단계	모든 arc 반복 전파
시점	변수 배정마다 1회	변수 배정마다 반복
효과	즉각 domain 정리	더 넓은 전파, 더 강한 pruning
한계	이웃의 이웃 불검사	pairwise만, global 불일치 미감지

FC vs AC-3 비교

Approximate Search: Beam Search p.23-29

Q. beam width K의 역할은?

Q. 왜 approximate인가? 복잡도?

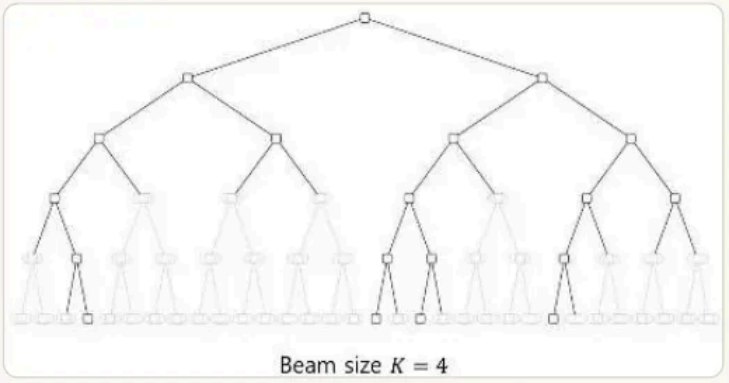
Backtracking vs Beam Search 대비 — Backtracking = *DFS* 정확 탐색. Beam = *pruned BFS* 근사 탐색.

- **Greedy search** (K=1): 각 단계마다 weight 최고인 값 1개만 유지. O(n·|D|)
- **BFS tree search** (K=∞): 모든 partial assignment 유지. O(|D|^n)
- **Beam search** (K 고정): 상위 K개 partial assignment만 유지 — **효율·정확 균형**

Beam Search (beam size K)

```
Initialize C ← [{}]                    ← 빈 partial assignment 1개
for i = 1, …, n:
  Extend:
    C' ← { a ∪ {X_i : v} : a∈C, v∈Domain(X_i) }
  Prune:
    C ← top-K elements of C' by weight   ← sort O(|C'| log |C'|)
return argmax_{a∈C} Weight(a)
```

K=1 → greedy. K=∞ → BFS. 시간: O(n·|D|·K·log(|D|·K)).



Beam size K=4: 각 레벨에서 상위 K만 생존 (p.27)

Beam Search 복잡도 & 특성

K 조정으로 효율·정확 트레이드오프

- 시간: **O(n·|D|·K·log(K·|D|))** — 정렬 비용 포함
- K=1 greedy = O(n) (branching factor |D|로 선형)
- K=∞ BFS = O(|D|^n) (지수)
- **최적해 미보장** — K가 클수록 정확, 작을수록 빠름
- Backtracking:DFS = Beam:BFS (구조적 유사)

★ **필수암기** Backtracking=exact·지수 / Beam=approximate·선형. K=1(greedy) → O(n·|D|). K=∞(BFS) → 지수. Beam: DFS 대응BFS, 순서는 weight 내림차순.

Approximate Search: Local Search (ICM) p.31-36

Q. ICM은 무엇을 반복 수정?

Q. 수렴 보장·전역최적 보장 여부?

Local Search 아이디어 — Backtracking·Beam은 *partial assignment를 확장*. Local search는 **완전 배정을 수정**: 한 번에 변수 하나씩 re-assign.

- 초기 완전 배정(랜덤)에서 시작 → 반복 개선
- Locality 활용: X_i 재배정 시 *X_i에 의존하는 factor만* 다시 계산

ICM (Iterated Conditional Modes) — 완전 배정 유지하며 변수를 순환 방문, 각 변수를 **조건부 최대화**.

단조증가 → 수렴
전역최적 보장 X

- 한 단계: $X_i = \text{argmax}_v \prod_{f: X_i \in \text{scope}(f)} f(a[-X_i] \cup \{X_i:v\})$
- **수렴 보장**: 각 업데이트 후 Weight(a) 단조 증가 또는 유지 → 유한 반복 후 수렴
- **지역 최적(local optimum)**에 수렴 가능 — 전역 최적 미보장

ICM (Iterated Conditional Modes)

```
Initialize a ← random complete assignment
repeat until convergence:
  for i = 1, …, n:
    for each value v in Domain(X_i):
      compute Weight(a ∪ {X_i : v})
    a[X_i] ← argmax_v Weight(a ∪ {X_i : v})   ← conditional mode
return a
```

Weight(a) 단조 비감소 → 유한 수렴. 단, local optimum 가능.

⚠ **함정** ICM은 **수렴은 보장**하지만 **최적해는 미보장**. Local optimum 탈출: 다중 랜덤 초기화(restarts) 또는 Gibbs sampling(확률적 버전).

알고리즘 총 비교 p.23-37

Q. 확장형 vs 수정형 구분은?

Q. exact vs approximate 선택 기준?

알고리즘	방식	시간복잡도	정확성	특징
Backtracking	partial 확장 (DFS)	$O(\|D\|^n)$	✓ Exact	MCV·LCV·FC·AC-3로 가지치기
Beam search	partial 확장 (BFS)	$O(n\|D\|K \log K\|D\|)$	✗ Approximate	K=1 greedy, K=∞ BFS
ICM	complete 수정	$O(n\cdot\|D\|\cdot\text{iter})$	✗ Approximate	수렴보장, local opt 가능
Gibbs sampling	complete 수정 (확률)	$O(n\cdot\|D\|\cdot\text{iter})$	✗ Approximate	ICM 확률적 버전

Factor graph 최대 가중치 배정 알고리즘 비교

★ **필수암기** (1) **Partial 확장**: Backtracking(exact·지수) · Beam(approx·선형). (2) **Complete 수정**: ICM(approx·결정론) · Gibbs(approx·확률론). AC-3 후에도 global inconsistency 미감지 가능 — arc≠global.

요약 Backtracking(MCV·LCV·FC·AC-3 가지치기, exact·지수) / Beam(approximate·선형·K) = **partial assignment 확장** · ICM = **complete assignment 반복 수정**(수렴 O·전역최적 X) · AC-3는 arc-consistent여도 global inconsistency 미감지.

Ch 13 Markov Networks — 마르코프 네트워크

factor graph + 확률 정규화 · Gibbs 샘플링 · 조건부 독립 · Markov blanket

Markov Network 정의 p.7-9

Q. factor graph와의 차이 (정규화 Z)?

Q. P(x)=Weight(x)/Z의 의미는?

Q. partition function Z란?

Markov Network (마르코프 네트워크) — factor graph에 **확률 정규화**를 더한 것. 랜덤 변수 $X=(X_1,\dots,X_n)$ 위의 결합 분포를 정의.

Weight → 확률 ÷ Z로 정규화

- 정의: P(X=x) = Weight(x) / Z — Z는 partition function (정규화 상수)
- Z = Σ_x Weight(x), $\texttt{Weight}(x) = \prod_i f_i(x_i)$
- Weight는 *음수 불가*(각 factor ≥ 0); Z > 0이면 유효 분포
- CSP와 달리 최대 가중치 배정 대신 주변 확률 $P(X_i=v)$를 계산

$$\mathbb{P}(X = x) = \frac{\text{Weight}(x)}{Z}$$

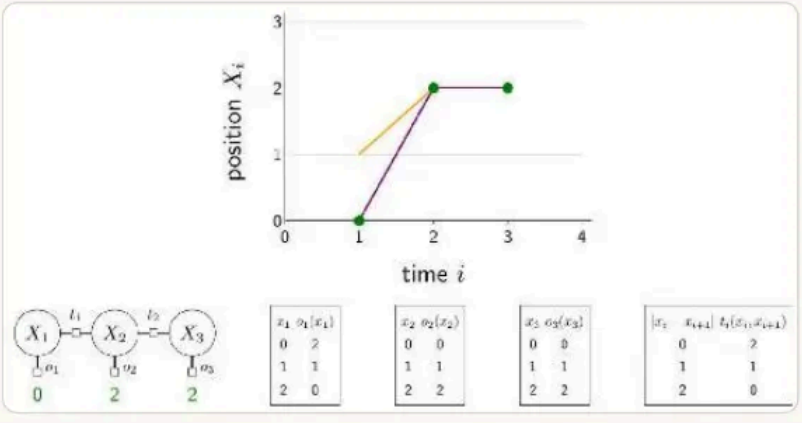
x_1	x_2	x_3	Weight(x)	$\mathbb{P}(X=x)$
0	1	1	4	0.15
0	1	2	4	0.15
1	1	1	4	0.15
1	1	2	4	0.15
1	2	1	2	0.08
1	2	2	8	0.31

$$Z = 4 + 4 + 4 + 4 + 2 + 8 = 26$$

Markov network 정의 수식 + 확률 분포표 (p.8)

Object Tracking 예제 — 위치 $X_1,X_2,X_3 \in \{0,1,2\}$, 관측 $o_1=0, o_2=2, o_3=2$.

- observation factor: $o_i(x_i)$ — 관측값과 일치할수록 높은 가중치
- transition factor: $t_i(x_i, x_{i+1}) = |x_i - x_{i+1}| + 1$ 의 역수에 비례 — 인접 위치 선호
- 최대 가중치 배정 {X_1:1,X_2:2,X_3:2}, Z=26
- $P(X_1=1) = 0.62 \neq$ argmax → 주변 확률 ≠ max 배정



Object tracking factor graph + 팩터 테이블 (p.6)

주변 확률 (Marginal Probability) — 변수 $X_i=v$의 주변 확률: $P(X_i=v) = \Sigma_{x:x_i=v} P(X=x)$

- 전체 결합 분포를 X_i 이외 변수에 대해 *합산(marginalize)*
- 직접 계산은 지수적 — 효율적 방법: Gibbs sampling

Gibbs Sampling p.10-12 · 주변확률 근사 알고리즘

Q. Gibbs sampling 한 스텝은?

Q. 조건부 분포로 무엇을 추정?

Q. 수렴(stationary)의 의미는?

Gibbs Sampling 아이디어 — 완전 배정을 무작위 초기화 후, 변수를 하나씩 **조건부 분포로 재샘플링**. 충분히 반복하면 카운트가 *실제 주변 확률로 수렴*.

- 조건부: $P(X_i=v \mid X_{-i}=x_{-i}) \propto$ `Σ_f Weight(f factors containing Xᵢ)`
- 수렴 충분조건: 모든 가중치 > 0 (기술적 조건)
- 수렴 속도: **지수 시간이 걸릴 수 있음** (보장 없음)

Gibbs Sampling

```
Initialize X ← random complete assignment
for t = 1 to T do
  for each variable Xᵢ do
    sample Xᵢ ~ P(Xᵢ | X_{-i} = x_{-i})    ← 나머지 고정
    count[i][xᵢ] += 1
Estimate P(Xᵢ=v) ≈ count[i][v] / Σ_v' count[i][v']
```

한 변수 샘플링: 연결된 factor들의 가중치 합산 → 정규화. Object tracking 예: $P(X_2=0) \propto 2$, $P(X_2=1) \propto 4$, $P(X_2=2) \propto 4$ → prob 0.2/0.4/0.4

ICM vs Gibbs — Search vs Sampling — 같은 factor graph에서 목적이 다른 두 알고리즘.

	ICM (Iterated Conditional Modes)	Gibbs Sampling
목적	최대 가중치 배정 (MAP)	주변 확률 $P(X_i=v)$
각 step	최고 가중치 값 선택	조건부 분포로 샘플링
수렴	지역 최적으로 수렴	올바른 주변확률로 수렴*
CSP 대응	Maximum weight assignment	Marginal probabilities

*모든 가중치 > 0 등 기술 조건 하. 둘 다 지수 시간 가능.

⚠ **함정** ICM은 local optimum에 빠진다 — 초기 배정에 따라 결과가 다름. Gibbs는 수렴하면 *전역적으로 올바른* 주변 확률을 복원하지만, 수렴이 **지수적으로 느릴 수 있다**.

Image Denoising 응용 — 픽셀 $X_i \in \{0,1\}$, 관측 $o_i(x_i)=[x_i=\text{관측값}]$, 인접 픽셀 factor $t_{ij}(x_i,x_j)=[x_i=x_j]+1$.

- Gibbs: 각 픽셀을 *나머지 픽셀 고정* 하에 조건부 샘플링
- 예: v=0 → weight=2, v=1 → weight=8 → $P(X_i=1 \mid \text{rest})=0.8$

독립성 & 조건부 독립 p.15-23 · 그래프 구조 활용

Q. 조건부 독립 A⊥B|C 정의는?

Q. 경로 차단(separation) 판정은?

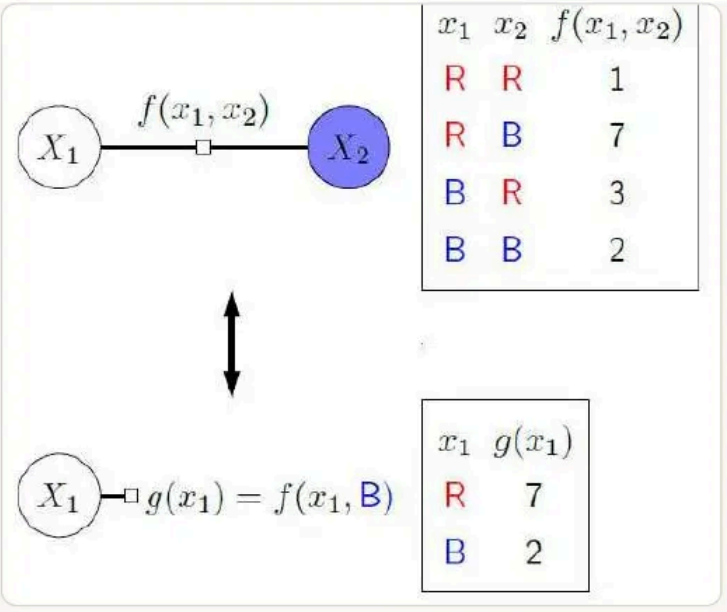
Independence (독립) — 변수 집합 A,B가 **연결 간선이 없으면** $A \perp B$.

- 그래프적: A와 B 사이에 *edge 없음* → 별도로 최적화 가능
- 예: 호주 지도 채색에서 Tasmania {T}는 나머지와 독립
- 독립이면 **분리해서 풀면 지수적 절감**

Conditioning (조건화) — 변수 $X_i=v$를 **고정** → X_i와 연결된 binary factor들을 unary factor로 교체 → 그래프 disconnected 유도.

변수 제거 → 그래프 분리

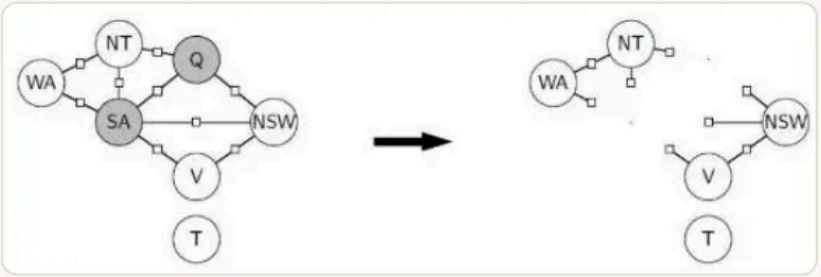
- 절차: $X_i=v$ 고정 → X_i에 연결된 모든 factor $f_1,\cdots,f_k$ 제거
- 새 factor 추가: `g_j(x_{-i}) = f_j(x_{-i}, v)` for $j=1\cdots k$
- 그래프적으로: X_i 노드와 연결 간선을 모두 *제거*



Conditioning: X_2=B 고정 → $f(x_1,x_2) \to g(x_1)$ (p.19)

Conditional Independence (조건부 독립) — A,B,C가 변수 분할. C에 조건화했을 때 A와 B가 독립이면 $A \perp B \mid C$.

- 동치 조건: A에서 B로 가는 *모든 경로*가 C를 통과
- 그래프적 검사: C를 제거했을 때 A-B 사이 경로가 없으면 성립



지도 채색: {WA,NT} ⊥ {V,NSW,T} | {SA,Q} (p.23)

Markov Blanket

p.24-26 · 최소 분리 집합

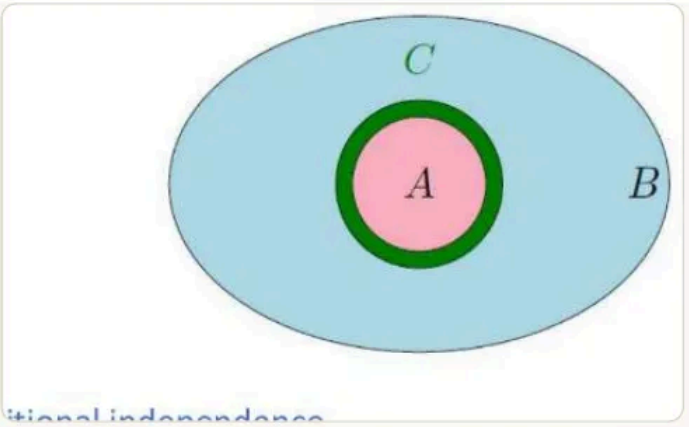
Q. Markov blanket의 정의는?

Q. 최소 분리 집합인 이유는?

Markov Blanket 정의 — 변수 부분집합 A ⊆ X의 MarkovBlanket(A) = A에 인접하지만 A 밖에 있는 모든 이웃.

A의 직접 이웃만 조건화하면 충분

- 형식: MarkovBlanket(A) = {v ∉ A | v와 A 사이에 edge 존재}
- C = MarkovBlanket(A), B = X \ (A ∪ C) 로 두면 $A \perp B \mid C$
- C는 A를 나머지로부터 *독립시키는 최소 집합*
- blanket이 작을수록 → factor graph 다루기 쉬움



Markov Blanket: A(분홍) 주위 C(녹색 링)가 B(파랑)로부터 분리 (p.25)

Markov Blanket 활용 — 지도 채색 — SA를 조건화하면 WA, NT, NSW, V가 서로 독립 → *각각 분리해서 최적화.*

- SA ∈ {R,G,B} 각 값에 대해: SA=v 고정 → 나머지 분리 → **각 부분 독립 최적화**
- 시간: 조건화 없이 3^n → 조건화 후 $3^1 \times$ (작은 서브그래프들)

> ✎ **핵심** Markov Blanket은 임의 노드 집합 A를 나머지 전체로부터 조건부 독립시키는 **최소 분리 집합**. $A \perp B \mid C$ (C=MarkovBlanket(A)) — 이 명제가 factor graph 효율화의 핵심.

CSP vs Markov Network 비교 p.13

Q. 0/1 팩터 vs 실수 팩터?

Q. 탐색 vs 확률추정 목적 차이?

개념	CSP (Constraint Satisfaction)	Markov Network
변수	Variables	Random Variables
가중치	Weights (Π factors)	P(X=x) = Weight(x)/Z
목표	Max weight assignment	Marginal $P(X_i=v)$
알고리즘	ICM (Iterated Cond. Modes)	Gibbs Sampling
수렴	Local optimum	Correct marginals*

Markov network = factor graph + 확률 정규화 (÷Z)

> ★ **필수암기** Markov network: P(X=x)=Weight(x)/Z, Z=partition function. Gibbs sampling: 각 변수를 조건부 분포로 재샘플링→주변확률 추정. A⊥B|C: C가 A→B 모든 경로 차단. MarkovBlanket(A)=A의 직접 이웃(A 제외)=최소 조건화 집합.

요약 Markov network = factor graph + **P(x)=Weight(x)/Z** 정규화 · Gibbs sampling으로 주변확률 추정 · A⊥B|C = C가 A-B 모든 경로 차단 · **Markov Blanket(A)=A를 분리하는 최소 이웃집합**.

Ch 14 Bayesian Networks 1 — 베이즈넷 기초

확률 복습 · BN 정의 · 확률적 추론 · Explaining Away · 확률적 프로그램

확률 복습 & Probabilistic Inference p.7-11

Q. joint·marginal·conditional 관계?

Q. inference query P(Q|E)란?

Q. Bayes rule의 형태는?

핵심 확률 법칙 — 모든 BN 추론의 수식 토대.

Chain rule이 BN 곱 공식의 기원

- **Joint probability**: P(A,B) = P(A∩B)
- **Conditional probability**: P(A|B) = P(A,B) / P(B)
- **Bayes rule**: P(A|B) = P(B|A)·P(A) / P(B)
- **Chain rule**: $P(X_1,\cdots,X_n) = \Pi_i P(X_i \mid X_1,\cdots,X_{i-1})$
- **Marginalization**: $P(X) = \Sigma_y P(X,Y=y)$ — 합산으로 숨은 변수 제거
- **Independence**: P(X,Y) = P(X)·P(Y)

Probabilistic Inference — 구조

- **Joint distribution**: 전 변수 결합 확률 $P(X_1,\cdots,X_n)$ — '확률적 데이터베이스'
- **Evidence** E=e: 관측된 변수 집합에 값 고정
- **Query** Q: 알고 싶은 변수 집합
- **Hidden** H: evidence도 query도 아닌 나머지 — **marginalize out**
- 출력: `P(Q | E=e)` — 사후 분포

> ✎ **핵심** 도전 과제: **n개 변수 joint = 크기 지수**적. BN은 **조건부 독립**을 이용해 compact하게 표현 → joint를 local CDP 곱으로 분해.

Bayesian Network 정의 p.12-14, 17-18

Q. BN factorization 식은?

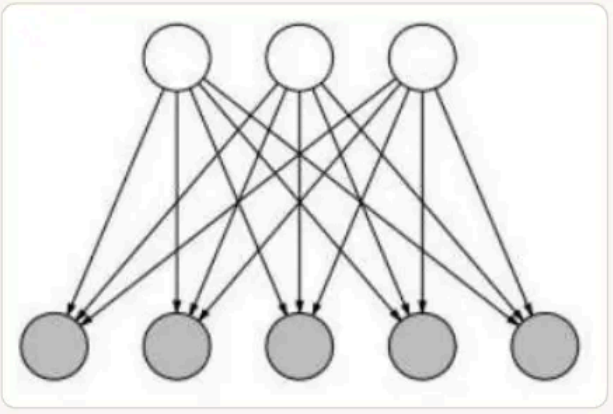
Q. DAG + local CPD의 의미는?

Q. joint를 어떻게 분해?

Bayesian Network (BN) — 변수들의 **조건부 독립 구조**를 DAG로 인코딩한 확률 모델.

LCD 하나 = factor graph의 하나의 factor

- 노드 = 확률 변수, 유향 간선 = 직접 영향(부모→자식)
- 각 노드 X_i에 **local conditional distribution** $p(x_i \mid x_Parents(i))$ 부여
- **Joint = LCD들의 곱**: $P(X_1=x_1,\cdots,X_n=x_n) = \prod_i p(x_i \mid x_Parents(i))$
- DAG ⟹ 사이클 없음 → topological order가 존재

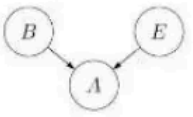


BN 구조 — DAG (p.17)

Alarm 예제 — BN 구체화 — 변수: B(burglary), E(earthquake), A(alarm). 구조: B→A, E→A.

- p(b): B의 prior (파라미터 ε)
- p(e): E의 prior (파라미터 ε)
- p(a|b,e): CPT — `p(a|b,e) = [a = (b ∨ e)]` (OR 게이트)
- Joint: $P(B=b,E=e,A=a) \equiv p(b)\cdot p(e)\cdot p(a|b,e)$

$$\mathbb{P}(B=b,E=e,A=a) \equiv p(b)p(e)p(a|b,e)$$



b	$p(b)$
1	ϵ
0	$1-\epsilon$

e	$p(e)$
1	ϵ
0	$1-\epsilon$

b	e	a	$p(a\|b,e)$
0	0	0	1
0	0	1	0
0	1	0	0
0	1	1	1
1	0	0	0
1	0	1	1
1	1	0	0
1	1	1	1

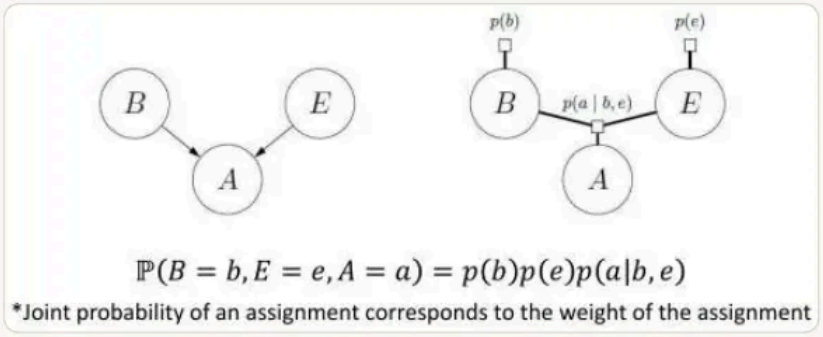
$$p(b) = \epsilon\cdot[b=1] + (1-\epsilon)\cdot[b=0]$$
$$p(e) = \epsilon\cdot[e=1] + (1-\epsilon)\cdot[e=0]$$
$$p(a|b,e) = [a = (b \vee e)]$$

*A local conditional distribution p corresponds to a factor

Alarm BN — DAG + CPT 전체 (p.12)

BN vs Factor Graph

- Factor graph: 변수 노드 + factor 노드(사각형) → 모든 인수의 곱
- BN: local conditional distribution = factor의 특수 케이스
- 결론: BN ⊂ Factor Graph — BN은 factor graph의 특수한 경우
- Assignment의 joint prob = assignment의 weight (factor graph 해석)



$$\mathbb{P}(B=b,E=e,A=a)=p(b)p(e)p(a|b,e)$$

*Joint probability of an assignment corresponds to the weight of the assignment

Alarm BN ↔ Factor Graph 대응 (p.13)

Probabilistic Inference & Explaining Away p.9-11, 14-16

Q. explaining away 현상은?

Q. v-structure에서 의존 발생 조건?

Q. 조건화 전후 독립성 변화?

Alarm 예제 — 추론 계산

marginalize로 E 또는 B를 합산 제거

- P(A=1) = ε(1−ε) + ε = 2ε−ε² (B=1 or E=1일 때 A=1)
- P(B=1|A=1) = ε/(2ε−ε²) → 알람이 울리면 burglary 의심↑
- P(B=1|A=1,E=1) = ε²/(ε)·… → 지진이 밝혀지면 burglary 의심↓

Explaining Away — 설명 소거 — 두 원인이 하나의 효과를 양의 방향으로 유발할 때, 효과를 조건화한 상태에서 한 원인이 밝혀지면 다른 원인의 확률이 감소한다.

- P(B=1|A=1) > P(B=1|A=1, E=1) — E가 A를 '설명'해 버리면 B는 덜 필요
- 중요: 두 원인 B, E는 서로 독립임에도 A를 조건화하면 조건부 의존 발생
- 의학 진단: 기침+가려운 눈 → 감기 or 알레르기? 하나가 확인되면 다른 하나 확률 감소

⚠ **함정** Explaining away는 원인 변수들이 독립이더라도 발생한다 — 조건화(conditioning)가 독립성을 깨뜨림. 'Effect를 관측'하면 원인들 사이에 v-structure 의존이 생김.

시나리오	P(B=1|…)
A=1 만 관측	ε/(2ε−ε²) > ε
A=1, E=1 관측	ε² / ε = ε (원래 prior와 같음)

Alarm 예제: 지진(E=1)이 알람을 설명하면 burglary 확률 → prior로 복귀

Probabilistic Programs & 모델 분류 p.21-29

Q. 생성 스토리(generative)란?

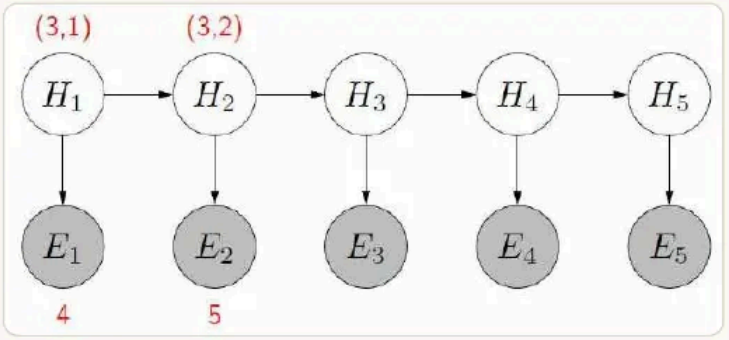
Q. HMM·Naive Bayes·LDA 분류 기준?

Probabilistic Program — BN을 랜덤화 프로그램으로 표현. 프로그램이 실행되면서 확률 변수들을 샘플링.

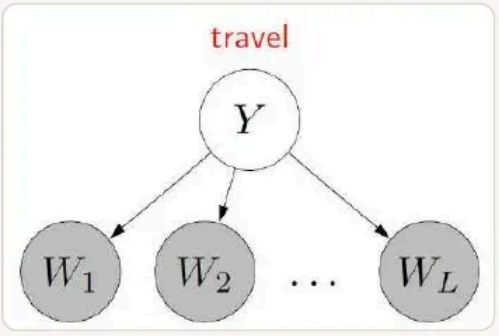
- Bernoulli(ε): random() < ε이면 1, 아니면 0
- 프로그램 구조 → BN 구조 (반복문 = 체인/격자 구조 생성)
- 패러다임: 생성 스토리 — 잠재 변수(출력)가 관측(입력)을 생성
- 일반 분류(입력→출력)와 반대 방향!

모델	Probabilistic Program 구조	BN 구조	Inference 질문
Markov Model	For t: $W_t \sim p(W_t \mid W_{t-1})$	$X_1 \to X_2 \to \cdots \to X_n$ 체인	문장 확률 스코어링
HMM	For t: $H_t \sim p(H \mid H_{t-1})$, $E_t \sim p(E \mid H_t)$	H 체인 + E 관측 레이어	센서→실제 위치
Factorial HMM	For t,k: $H_t^k \sim p$, $E_t \sim p(E \mid H\text{^}a, H\text{^}b)$	두 H 체인 + 공유 E	복수 객체 추적
Naive Bayes	$Y \sim p(Y)$; $W_i \sim p(W \mid Y)$	Y 루트 → $W_1 \cdots W_L$	문서→카테고리
LDA	$\alpha \sim \text{Dir}$; $Z_i \sim p(Z \mid \alpha)$; $W_i \sim p(W \mid Z_i)$	토픽 혼합 + 단어	문서→토픽 분포

주요 BN 모델 비교 — 생성 스토리 구조 차이



HMM BN — H 체인 + 관측 E (p.24)



Naive Bayes BN — Y가 모든 W 생성 (p.26)

★ 필수암기 BN = DAG + local conditional distributions 곱. Joint = Π_i p(x_i|x_Parents(i)). Explaining away: 독립 원인도 effect 조건화하면 의존. BN ⊂ Factor Graph. Probabilistic program → 생성 스토리 → BN 구조 자동 결정.

요약 BN = DAG + local conditional 곱 Π_i p(x_i|x_Parents(i)) · Explaining away: 독립 원인도 공통 effect 조건화하면 v-structure 의존 발생 · probabilistic program → 생성 스토리 → 모델 분류(HMM·NB·LDA).

Ch 15 Bayesian Networks 2 — HMM 추론

일반 추론 전략 · Forward-Backward · Particle Filtering

Bayesian Network & Inference 리뷰 p.3-6

Q. BN 추론에서 묻는 것은?

Q. joint→query 환원 흐름은?

Bayesian Network 정의 — 랜덤변수 $X=(X_1,\cdots,X_n)$에 대한 DAG — 노드별 조건부분포의 곱으로 결합분포를 표현.

- 결합분포: $\mathbb{P}(X_1,\cdots,X_n) = \Pi_i\, p(X_i \mid \text{parents}(X_i))$
- Markov network 관계: Bayes net = Markov net with normalizer Z=1

확률적 추론 문제 — 입력: BN + evidence E=e + query Q. 출력: $\mathbb{P}(Q \mid E=e)$.

- 예시: 기침(H=1)·가려운눈(I=1) → 감기(C=c)일 확률 = $\mathbb{P}(C \mid H=1, I=1)$
- Evidence 조건화: 관측된 변수를 특정값으로 고정 → 잔여 네트워크만 추론

Markov network로 환원 — BN에서 evidence 조건화 후 Markov network 공식으로 변환.

- Z^{-1} 인자(정규화상수)를 곱해 사후분포 계산 가능
- **Unobserved leaves 제거**: 쿼리와 무관한 **관측되지 않은 leaf 노드**는 marginalize 후 버림 — 추론 전에 제거
- **Disconnected components 제거**: evidence·query와 **단절된 컴포넌트**는 독립 → 추론 전 버림

일반 확률적 추론 전략 p.7-10

Q. 추론 5단계 절차는?

Q. leaf·단절 노드 제거 근거는?

Q. Markov net으로 왜 변환?

General Probabilistic Inference Strategy

```
Input: BN ℙ(X₁,…,Xₙ),  evidence E=e,  query Q
1. Condition on evidence  (evidence 변수를 관측값으로 고정)
2. Throw away unobserved leaves  (비관측 leaf 노드 제거)
3. Throw away disconnected components  (단절 컴포넌트 제거)
4. Define Markov network out of remaining factors
5. Run inference algorithm  (예: manual, Gibbs sampling, …)
```

단계 2·3을 반복 적용해 네트워크를 최대한 축소한 뒤 추론 → 비용 절감

✎ **핵심** 단계 2: 관측 없는 leaf를 제거하면 그 부모의 인자가 marginalize-out됨. 단계 3: query·evidence와 **연결이 없는** 변수는 결과에 영향 없음 → 버려도 $\mathbb{P}(Q|E=e)$ 불변.

Hidden Markov Model (HMM) p.12-13

Q. transition·emission factor는?

Q. lattice(trellis) 구조란?

Q. filtering vs smoothing?

HMM 구조 (object tracking 예시) — 숨은 상태 $H_1,\cdots,H_n$ ↔ 관측 $E_1,\cdots,E_n$. 결합분포를 세 인자로 인수분해.

초기·전이·방출 세 인자

- $\mathbb{P}(H=h, E=e) = p(H_1) \cdot \prod p(H_i|H_{i-1}) \cdot \prod p(E_i|H_i)$
- **초기분포** $p(H_1)$ · **전이분포** $p(H_i|H_{i-1})$ · **방출분포** $p(E_i|H_i)$

HMM 추론 질문 유형

- **Filtering**: $\mathbb{P}(H_i \mid E_1=e_1, \cdots, E_i=e_i)$ — 현재까지 관측만 사용
- **Smoothing**: $\mathbb{P}(H_i \mid E_1=e_1, \cdots, E_n=e_n)$ — 미래 관측까지 전부 사용
- Filtering = Smoothing의 특수 경우 (미래 관측 → unobserved leaf → 제거)

Forward-Backward Algorithm p.14-16

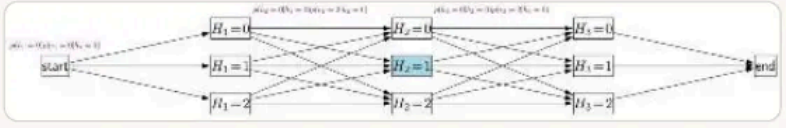
Q. forward F_i·backward B_i 정의?

Q. marginal을 어떻게 합성?

Q. 복잡도 $O(n|D|^2)$ 구성은?

Lattice 표현 — 변수 기반 BN → **상태 기반 격자** (각 상태 = 변수의 특정 값)로 변환.

- Edge start → $H_1=h_1$의 가중치 = $p(H_1=h_1)\cdot p(E_1|H_1=h_1)$
- Edge $H_i=h_i \to H_{i+1}=h_{i+1}$의 가중치 = $p(h_i|h_{i-1})\cdot p(e_i|h_i)$
- **start→end 경로 하나 = 하나의 complete assignment** (가중치 = 경로 상의 모든 edge weight의 곱)
- $\mathbb{P}(H_i=h_i \mid E=e)$ = $H_i=h_i$를 지나는 경로의 가중합 비율

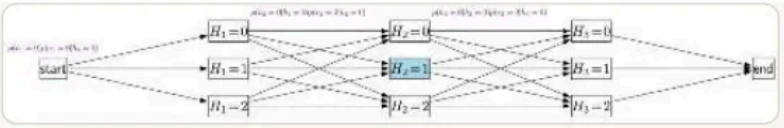


Lattice 표현: 각 열이 H_i의 값들, 경로=assignment (p.14)

Forward·Backward 메시지 — 경로 합산을 동적프로그래밍으로 효율화 — 두 방향 메시지 정의.

F: 좌→우
B: 우→좌
S: 곱

- Forward $F_i(h_i) = \Sigma_{\{h_{i-1}\}} F_{i-1}(h_{i-1})\cdot \text{Weight}(H_{i-1}=h_{i-1}, H_i=h_i)$
- → **start부터 $H_i=h_i$까지의 경로 가중합**
- Backward $B_i(h_i) = \Sigma_{\{h_{i+1}\}} B_{i+1}(h_{i+1})\cdot \text{Weight}(H_i=h_i, H_{i+1}=h_{i+1})$
- → **$H_i=h_i$부터 end까지의 경로 가중합**
- $S_i(h_i) = F_i(h_i)\cdot B_i(h_i)$ — start부터 end까지 *$H_i=h_i$를 통과하는* 경로 가중합



Forward/Backward 메시지 방향 (p.15)

Forward-Backward Algorithm

```
# Step 1: Forward pass (left → right)
Compute F₁, F₂, …, Fₙ
  F₁(h₁) = p(h₁)·p(e₁|h₁)
  Fᵢ(hᵢ) = Σ_{h_{i-1}} F_{i-1}(h_{i-1})·p(hᵢ|h_{i-1})·p(eᵢ|hᵢ)
# Step 2: Backward pass (right → left)
Compute Bₙ, B_{n-1}, …, B₁
  Bₙ(hₙ) = 1
  Bᵢ(hᵢ) = Σ_{h_{i+1}} B_{i+1}(h_{i+1})·p(h_{i+1}|hᵢ)·p(e_{i+1}|h_{i+1})
# Step 3: Combine and normalize
for each i do
  Sᵢ(hᵢ) = Fᵢ(hᵢ)·Bᵢ(hᵢ)
  ℙ(Hᵢ=hᵢ | E=e) = Sᵢ(hᵢ) / Σ_v Sᵢ(v)      ← normalize
```

시간복잡도: O(n·|Domain|²) — DP로 공유 중간 계산. 모든 smoothing 질문을 한 번에 답변 가능.

★ 필수암기 Forward-Backward: F = start→h_i 경로합, B = h_i→end 경로합, S = F·B = h_i를 통과하는 전체 경로합. 정규화: $\mathbb{P}(H_i{=}h_i|E{=}e) = S_i(h_i) / \Sigma_v S_i(v)$. 복잡도 O(n·|Domain|²).

Particle Filtering p.20-28

Q. Propose·Weight·Resample 3단계?

Q. 왜 근사이며 언제 유리?

Q. 복잡도 O(nK)의 의미?

Forward-Backward의 한계 — 실수 위치값 등 |Domain| 크면 O(n·|Domain|²)이 너무 느림.

- Beam search: K개 최고 가중치 파티클 유지 — 탐욕적 prune이 *다양성* 파괴
- → Particle Filtering: propose→weight→resample 3단계로 해결

Particle Filtering 3단계 직관 — 파티클 집합으로 근사분포를 표현하고, 새 관측이 올 때마다 갱신.

Propose Weight Resample

- ① Propose: 전이분포 $p(h_i|h_{i-1})$으로 각 파티클에서 다음 상태 샘플링
- ② Weight: 방출분포 $p(e_i|h_i)$로 파티클에 가중치 부여
- ③ Resample: 가중치 비례로 K개 재추출 → 유망한 영역으로 재집중

Particle Filtering

```
Initialize  C ← [{}]    ← K개 빈 파티클
for i = 1, …, n do
  # Propose: 전이분포로 다음 상태 샘플링
  C' ← { h ∪ {Hᵢ: hᵢ} : h ∈ C, hᵢ ~ p(hᵢ | h_{i-1}) }
  # Weight: 관측 eᵢ에 의한 가중치 계산
  w(h) = p(eᵢ | hᵢ)  for each h ∈ C'
  # Resample: 가중치 비례로 K개 독립 추출
  C ← K particles drawn ∝ w(h) / Σ_{h'∈C'} w(h')
# 출력: normalize weights → 근사 ℙ(H₁,…,Hₙ | E=e)
```

filtering 질문: 마지막 H_i값만 유지해도 됨. 동일 파티클은 count로 압축 가능.

단계	핵심 아이디어	사용 분포
Propose	전이분포로 미래 상태 샘플 → 공간 탐색	$p(h_i \mid h_{i-1})$
Weight	관측 일치도로 파티클 점수 → 증거 반영	$p(e_i \mid h_i)$
Resample	가중치 비례 K개 재추출 → 다양성 유지	∝ weight

Particle Filtering 3단계 — Beam search(greedy prune)와 달리 확률적 재추출로 다양성 보존

Beam search vs Particle Filtering 비교

- Beam search: extend(모든 값 탐색=느림) + prune(greedy=다양성 손실)
- Particle Filtering: propose(전이분포 샘플=효율적) + resample(확률적=다양성 유지)
- Particle Filtering: |Domain| 무관 — 대규모 연속 공간으로 확장 가능

⚠ **함정** Particle Filtering은 근사 알고리즘 — K(파티클 수)가 작으면 분산이 커진다. Forward-Backward와 달리 정확한 답을 보장하지 않음.

★ **필수암기** Forward-Backward: 정확·O(n|Domain|²) → 소규모 이산 도메인. Particle Filtering: 근사·O(nK) → 대규모·연속 도메인. 3단계 키워드: Propose(전이분포 샘플)·Weight(방출분포 가중)·Resample(비례 재추출).

요약 BN 추론 5단계(evidence 조건화→leaf 제거→단절 제거→Markov net 변환→알고리즘) · HMM: **Forward-Backward**(lattice DP, O(n|D|²), 정확) vs **Particle Filtering**(Propose→Weight→Resample, O(nK), 근사·대규모).

Ch 16 Bayesian Networks 3 — 파라미터 학습

Supervised Learning · Maximum Likelihood · Laplace Smoothing · EM Algorithm

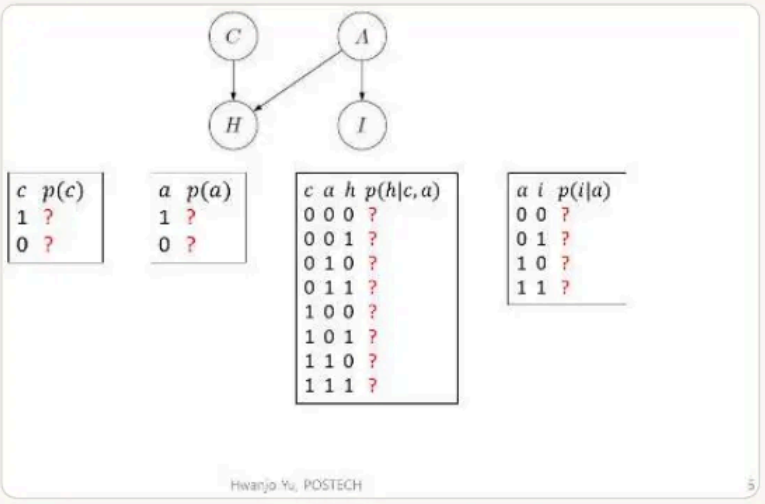
학습 과제 (Learning Task) p.5-6

Q. parameter learning이 묻는 것은?

Q. 완전관측 vs 부분관측 데이터?

파라미터 학습이란 — BN 구조(DAG)는 주어진 상태에서, 각 노드의 local conditional distribution θ를 훈련 데이터로부터 추정하는 문제.

- **입력**: 완전 배정(full assignments)의 훈련 데이터 `D`
- **출력**: 파라미터 집합 `θ = {p_d : d ∈ D}` (e.g., Genre, Rating)
- 각 변수 `x_i` 는 분포 `p_{d_i}` 에서 생성 — parameter sharing 가능(여러 변수가 같은 θ 사용)



BN 구조와 CPT 예시 — 각 노드마다 물음표 파라미터를 추정 (p.5)

✎ **핵심** 파라미터 수: 루트 노드는 값 수 −1, 내부 노드는 (부모 조합 수) × (값 수 −1). Naïve Bayes — `2 + 2·|V|` params; HMM — `|S| + |S|² + |S|·|O|` params.

Maximum Likelihood — Count & Normalize p.7-22

Q. MLE의 count & normalize 절차?

Q. closed-form인 이유는?

직관 — 비율로 추정 — 각 조건부 확률은 해당 조합이 데이터에 등장한 횟수의 비율.

- 단일 변수: `p(x=v) ∝ count(v)` — 전체 N개 중 v 등장 횟수
- 조건부: `p(x_i | x_Parents(i)) ∝ count(x_Parents(i), x_i)`
- 각 분포를 *독립적으로* 추정 가능 → 문제가 decompose됨

Count & Normalize (Maximum Likelihood)

```
Input:  D  (full assignments), θ = {p_d : d ∈ D}
Output: θ (ML parameters)
# Count phase
for each x ∈ D:
  for each variable x_i:
    Increment count_d_i(x_Parents(i), x_i)
# Normalize phase
for each d and local assignment x_Parents(i):
  Set p_d(x_i | x_Parents(i)) ∝ count_d(x_Parents(i), x_i)
```

각 분포 d를 독립 subproblem으로 풀 수 있다. 로그를 취해 미분하면 closed-form 해가 나옴.

Maximum Likelihood 목적함수 — 모든 데이터의 결합 확률을 최대화하는 θ 탐색.

- 목적: `max_θ ∏_{x∈D} P(X=x ; θ)`
- 분해: `max p_G · max p_R|G · ...` — 각 분포마다 독립 최적화
- 해: closed-form — count and normalize가 정확히 MLE

⚠ **함정** MLE 문제점 — 데이터에 한 번도 등장하지 않은 값/조합에 대해 확률 0 배정 → *Overfitting*. 예: 훈련에 (comedy, 2) 없으면 p_R(2|c)=0, 실제론 가능한 값인데 완전히 배제됨.

Laplace Smoothing p.24-28

Q. 0-확률 문제와 +λ의 효과?

Q. λ가 추정에 미치는 영향은?

핵심 아이디어 — 가상 카운트 추가 — 각 조합마다 λ개의 가상(hallucinated) 데이터를 추가해 0-확률을 방지.

λ=1: add-one
λ→∞: uniform

- 수식: `count_d(x_Parents, x_i) ← count_d(...) + λ`
- λ=1이 가장 흔함 ("add-one smoothing")
- 분모도 λ × (값 개수)만큼 증가 → 정규화 유지

λ 크기와 데이터의 상호작용 — λ가 클수록 균등분포에 가까워지고, 데이터가 많을수록 smoothing이 희석됨.

- 소량 데이터: λ가 확률을 dominant → 근사 균등분포
- 대량 데이터: data wins out → MLE에 수렴 (1+998 ≈ 0.999)

케이스	count(d)	count(c)	p(d)	p(c)
λ=1/2, n=1개	1/2+1	1/2	3/4	1/4
λ=1, n=1개	1+1	1+1	2/3 (?)	1/3 (?)
λ=1, n=999개	1+998	1	≈0.999	≈0.001

λ=1, 소량 데이터 vs 대량 데이터 — 데이터가 많으면 smoothing이 흡수됨 (p.27)

★ **필수암기** Laplace smoothing 공식: `p_d(x_i | x_Par) = (count(...) + λ) / (Σ count(...) + λ·|dom(x_i)|)` . λ=0이면 MLE. 모든 unseen 조합에 >0 확률 보장.

EM Algorithm — 미관측 변수 p.30-38

Q. E-step·M-step 각각 무엇?

Q. 왜 반복? 무엇이 단조증가?

Q. local optimum 수렴의 의미?

동기 — Hidden Variable — 일부 변수가 관측되지 않을 때 MLE(*count & normalize*)를 직접 적용 불가.

- 완전 관측: `D = {(d,4,5),(d,4,4),...}` → 바로 MLE
- Hidden G: `D = {(?,4,5),(?,4,4),...}` → Genre를 모름
- 목표: 최대 주변 우도(maximum marginal likelihood) = `max_θ Σ_h P(H=h, E=e ; θ)`

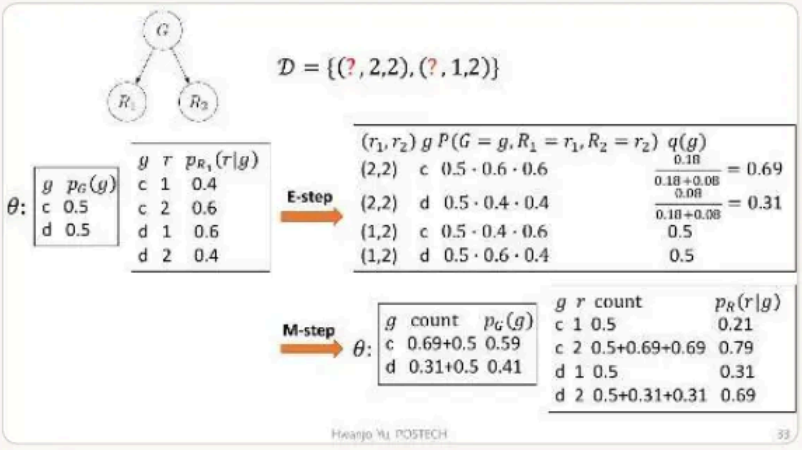
Expectation Maximization (EM)

```
Input:  observed data D = {e^(1), ..., e^(n)},  BN structure
Output: parameters θ
Initialize θ randomly
repeat until convergence:
  # E-step: soft-완성 (probabilistic inference)
  for each example e in D:
    for each hidden value h:
      q(h) = P(H=h | E=e ; θ)      ← 현재 θ로 추론
    Create weighted example (h, e) with weight q(h)
  # M-step: weighted MLE
  θ = count_and_normalize(weighted examples)      ← 분수 카운트
```

수렴 보장: 각 반복에서 marginal likelihood가 단조 증가. K-means는 EM의 특수 케이스 (hard E-step).

E-step vs M-step 역할

- E-step: 현재 θ로 hidden `H` 의 사후 분포 `q(h)` 계산 → 소프트 완성(soft completion) — 각 예제가 여러 hidden 값에 분산
- M-step: 가중 카운트(`fractional count`) → count & normalize → 새 θ
- 반복: θ → q(h) → θ → q(h) ... 수렴 (local optimum 보장)



EM 1 iteration — D={(?,2,2),(?,1,2)}, E-step 가중 배정 → M-step count & normalize (p.33)

⚠ **함정** EM은 local optimum에 수렴 — global 최적 보장 없음. 초기값 θ에 따라 결과 달라질 수 있음 → 여러 random init 권장.

응용 — 암호 해독 (Decipherment as HMM) p.34-38

Q. decipherment를 HMM으로 보는 법?

Q. EM이 여기서 하는 역할은?

치환 암호 → HMM으로 모델링 — 평문 문자열(`H_1,...,H_n`)이 hidden, 암호문(`E_1,...,E_n`)이 observed.

- 파라미터 `θ = (p_init, p_trans, p_emit)`
- p_init: 균등분포로 고정
- p_trans: 대량 영어 텍스트로 추정 (언어 모델)
- p_emit: 치환 테이블 — EM으로 추정

EM 적용

- E-step: forward-backward로 각 위치 `t` 와 문자 `h` 에 대해 `q_t(h) = P(H_t=h | E=e ; θ)` 계산
- M-step: `count_emit(h, c) = Σ_t q_t(h)·[E_t=c]` → `p_emit(c|h) ∝ count_emit(h, c)`
- 직관: *언어 모델이 영어다운 평문 h를 선호 + p_emit이 일관된 치환을 선호 → 수렴*

학습 알고리즘 비교 p.20-32

Q. MLE vs Laplace vs EM 적용 조건?

	MLE (Count & Normalize)	MLE + Laplace	EM
관측 조건	완전 관측	완전 관측	일부 hidden 가능
핵심 연산	count → normalize	count+λ → normalize	E-step + M-step 반복
0-확률	가능 (overfitting)	불가 (λ>0)	가능
수렴	Closed-form (1회)	Closed-form (1회)	반복, local optimum
복잡도	O(|D|·n)	O(|D|·n)	O(iter · inference)

세 학습 알고리즘 비교. EM은 hidden 변수 처리를 위해 반복 추론 필요.

★ 필수암기 MLE = count & normalize, closed-form, 완전관측 필수. Laplace = +λ hallucinated, 0-prob 방지, λ 크면 uniform 편향. EM = E-step(soft 추론)+M-step(weighted MLE) 반복, hidden variable 처리, local opt 수렴. K-means ⊂ EM (hard E-step).

요약 BN 파라미터 학습 · **MLE=count & normalize**(closed-form, 완전관측) · Laplace smoothing(+λ로 0-확률 방지) · **EM = E-step(soft 추론)+M-step(weighted MLE) 반복**으로 hidden var 처리 → local optimum 수렴.

Ch 17 Logic 1 — Propositional Logic

Syntax · Semantics · Knowledge Base · Entailment · Satisfiability

Logic-Based Models p.3-6

Q. logic 기반 모델의 목적은?

Q. syntax vs semantics 구분?

3가지 모델링 패러다임 — AI 모델의 스펙트럼 — 표현력 ↔ 계산 효율 트레이드오프.

- State-based — search, MDP, game: state·action·cost로 사고
- Variable-based — CSP, Bayes net: variable·factor로 사고
- Logic-based — propositional / first-order logic: formula + inference rules로 사고 · *High-level intelligence*

Logic의 두 목표 — ① 세계에 대한 지식을 표현 (Represent knowledge) · ② 그 지식으로 추론 (Reason with knowledge).

✎ 핵심 Logic의 핵심 트레이드오프: 표현력(expressivity) ↑ → 계산 효율 ↓. Horn clause 제한 propositional → 완전 propositional → first-order → second-order 순으로 표현력 증가.

Syntax of Propositional Logic p.16-18

Q. atomic·connective 구성요소는?

Q. formula 재귀적 정의는?

Syntax 정의 — 유효한 formula(수식)의 집합을 정의. 아직 의미(semantics) 없음 — 순수 기호.

- Propositional symbols (atomic): A, B, C, … — 더 이상 나눌 수 없는 명제 단위
- Logical connectives: ¬ (negation), ∧ (conjunction), ∨ (disjunction), → (implication), ↔ (biconditional)

Formula 재귀 구성 규칙 — f, g가 formula이면 ¬f, f∧g, f∨g, f→g, f↔g 도 formula (recursive definition).

- Formula 예: ¬A, ¬A→B, ¬A∧(¬B→C)∨(¬C∨D), ¬¬¬A
- Non-formula: ¬ (피연산자 없음), A+B ('+' 는 propositional connective 아님)

Semantics — Model & Interpretation p.19-25

Q. interpretation/model w란?

Q. I(f,w) 재귀 평가 방식은?

Q. f가 만족된다는 의미는?

Model w — 명제 기호에 진릿값을 대입한 완전 할당(assignment).

model = 세계의 한 상태

- n개 기호 → 2^n개 가능한 model
- 예: {A:1, B:1, C:0} — 하나의 model

Interpretation function I(f, w) — formula f와 model w를 받아 **true/false** 반환. 재귀 정의로 복합 formula를 원자부터 평가.

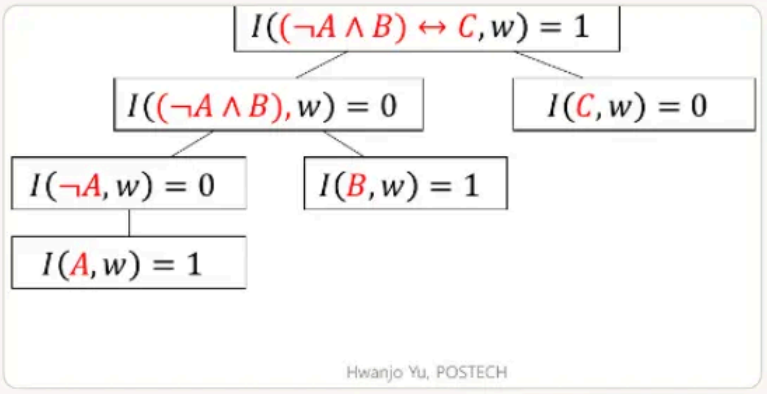
- Base: I(p, w) = w(p) ← 원자 기호는 model의 값을 그대로
- Recursive: 아래 진리표로 ¬f, f∧g, f∨g, f→g, f↔g 계산

I(f,w)	I(g,w)	I(¬f,w)	I(f∧g,w)	I(f∨g,w)	I(f→g,w)	I(f↔g,w)
0	0	1	0	0	1	1
0	1	1	0	1	1	0
1	0	0	0	1	0	0
1	1	0	1	1	1	1

진리표 — implication f→g: f=0이면 무조건 1 (vacuously true) (p.22)

Parse tree로 재귀 평가 (Worked Example) — f = (¬A∧B)↔C, w = {A:1, B:1, C:0}

- I(A,w)=1 → I(¬A,w)=0 → I(¬A∧B,w)=0∧1=0 → I(f,w)=I(0↔0,w)=1



Parse tree 재귀 평가 예시 — bottom-up으로 진릿값 전파 (p.23)

M(f) — formula가 나타내는 model 집합 — M(f) = {w | I(f,w)=1}. formula는 **model 집합을 컴팩트하게 표현**하는 수단.

- 예: f = (A∧B)↔C → M(f) = {{A:0,B:0,C:0}, {A:1,B:0,C:0}, {A:0,B:1,C:0}, {A:1,B:1,C:1}}

Knowledge Base (KB) p.26-28

Q. KB를 model 집합으로 보는 관점?

Q. KB가 model 교집합인 이유?

Knowledge Base 정의 — formula들의 집합 — 교집합(conjunction)으로 해석: M(KB) = ∩_{f∈KB} M(f). KB는 **세계에 대한 제약 조건** 집합.

- 예: KB = {Rain∨Snow, Traffic} → M(KB) = M(Rain∨Snow) ∩ M(Traffic)
- formula 추가 → M(KB) **축소** (더 강한 제약) — "더 많이 알수록 후보 세계가 줄어든다"

Entailment · Contradiction · Contingency p.29-32

Q. entailment KB⊨f 정의는?

Q. contradiction·contingency 구분?

Q. Venn 다이어그램 해석은?

Entailment — KB ⊨ f — KB가 f를 **함의**: M(KB) ⊆ M(f). f가 아무 정보도 추가 안 함 (이미 알려진 사실).

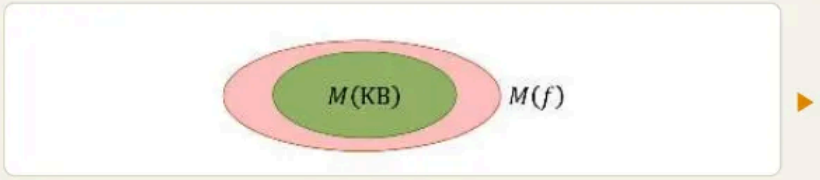
- 예: Rain∧Snow ⊨ Snow

Contradiction — KB가 f와 **모순**: M(KB) ∩ M(f) = ∅. f는 KB와 양립 불가.

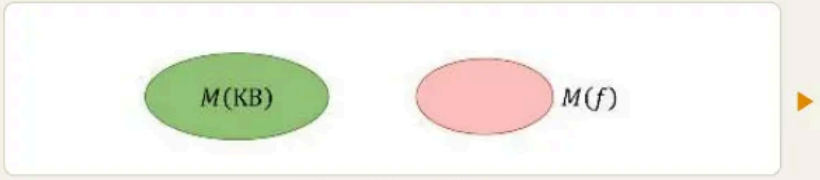
- 예: Rain∧Snow contradicts ¬Snow
- **명제**: KB contradicts f ⟺ KB ⊨ ¬f

Contingency — f가 KB에 **비자명 정보를 추가**: ∅ ⊂ M(KB)∩M(f) ⊂ M(KB).

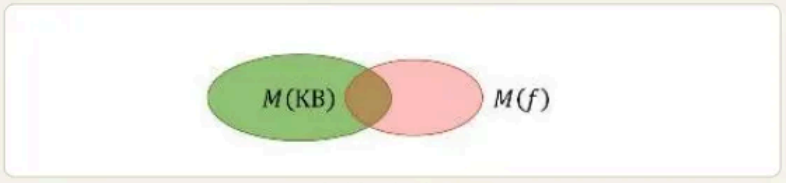
- 예: KB에 Rain이 있을 때 Snow는 contingent



① Entailment: M(KB)⊆M(f)



② Contradiction: 교집합=∅



③ Contingency: 부분 교집합

세 가지 관계 — Venn 다이어그램 비교 (p.29-31)

Tell & Ask Operations p.33-34

Q. Tell·Ask 연산의 의미는?

Q. 새 정보의 3가지 분류는?

Tell[f] — 정보 입력 — KB에 formula f를 알림. 세 가지 응답:

- Entailment: KB ⊨ f → 이미 알고 있음
- Contradiction: KB ⊨ ¬f → 믿지 않음
- Contingent: KB를 f로 업데이트 (새 정보 학습)

Ask[f] — 질의 — KB에 f를 질의. 세 가지 응답:

- Yes: entailment (KB ⊨ f)
- No: contradiction (KB ⊨ ¬f)
- 모름: contingent

> 💡 **TIP** Tell ↔ Ask의 대칭성: 같은 KB를 공유하며, 둘 다 entailment 판단으로 귀결됨. 핵심은 KB ⊨ f를 효율적으로 계산하는 것.

Satisfiability & Model Checking p.36-39

Q. SAT 문제 정의는?

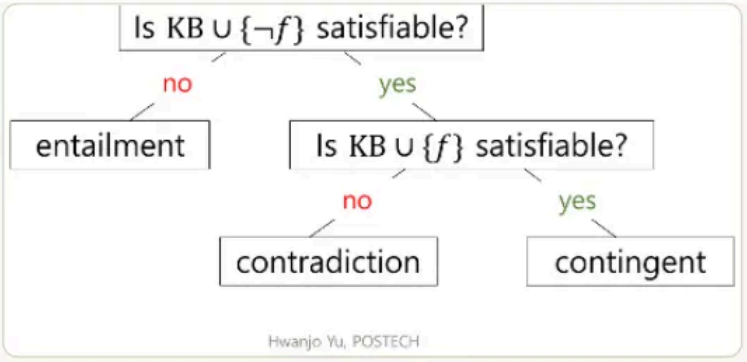
Q. Ask/Tell의 2-step SAT 환원?

Q. model checking 동작은?

Satisfiability 정의 — KB가 satisfiable: M(KB) ≠ ∅ (참인 세계가 하나 이상 존재).

Ask/Tell → SAT로 환원 — entailment/contradiction/contingency 판단을 모두 SAT 문제로 바꿀 수 있음.

- "Is KB∪{¬f} satisfiable?" 로 시작 → no: entailment / yes: 두 번째 질문으로
- "Is KB∪{f} satisfiable?" → no: contradiction / yes: contingent



Ask[f] → SAT 환원 결정 트리 (p.36)

Model Checking — SAT는 CSP의 특수 사례 — propositional symbol → CSP variable, formula → constraint, model → assignment.

- 대표 알고리즘: **DPLL** (backtracking + pruning), **WalkSAT** (randomized local search)
- SAT는 NP-complete — 최악 지수 시간이나 실용적 solver는 빠름

> ★ **필수암기** Syntax = 유효 formula 정의(기호, 의미 없음). Semantics = model w에서 I(f,w)∈{0,1} 계산(진리표+재귀). M(f)=formula를 만족하는 model 집합. KB ⊨ f ⟺ M(KB)⊆M(f). Contradiction ⟺ KB ⊨ ¬f. SAT: KB satisfiable ⟺ M(KB)≠∅. Ask/Tell 모두 SAT로 환원 가능.

요약 Propositional Logic: **Syntax(재귀 formula) → Semantics(I(f,w) 재귀평가) → KB(model 집합 교집합)** · Entailment/Contradiction/Contingency(Venn) · Ask/Tell → **SAT/model checking**으로 결정.

Ch 18 Logic 2 — First-Order Logic

명제 논리 추론 규칙 (Soundness · Completeness · Horn · Resolution) + FOL 소개

추론 프레임워크 p.4-6

Q. inference rule(⊢)의 역할은?

Q. derivation(증명)이란?

Inference Rule 정의 — 전제(premises) $\varphi_1,\cdots,\varphi_n$ 으로부터 결론 ψ 를 도출하는 구문적 규칙. 의미(semantics) 아닌 구문(syntax)에만 작동.

- **Modus ponens**: $\varphi,\ \varphi\to\psi \vdash \psi$ — 가장 기본적인 추론 규칙
- 일반형: $\varphi_1,\cdots,\varphi_n$ (premises) / ψ (conclusion)

Forward Inference (전방 추론)

```
Input: Rules, KB
repeat until no changes to KB:
  choose φ1,…,φn ∈ KB
  if matching rule φ1,…,φn / ψ exists:
    add ψ to KB
```

KB ⊢ ψ (derivation): ψ가 KB에 결국 추가되면 '유도됨(derives)'

Modus Ponens 예시

- KB = {Rain, Rain→Wet, Wet→Slippery}
- Step 1: Rain + Rain→Wet → Wet 추가
- Step 2: Wet + Wet→Slippery → Slippery 추가
- 한계: ¬Wet, Wet∧Slippery 는 도출 불가 — 규칙의 표현력 한계

Soundness & Completeness p.8-15 · 핵심 두 성질

Q. soundness {⊢}⊆{⊨} 의미는?

Q. completeness {⊢} ⊇{⊨} 의미는?

Q. 둘의 차이를 한 줄로?

Entailment vs Derivation — 의미론적 귀결: KB ⊨ φ (모든 model에서 true). 구문론적 유도: KB ⊢ φ (추론 규칙으로 도달). 두 집합의 포함 관계가 **soundness/completeness**를 정의.

sound → 거짓 없음
complete → 빠짐없음

- **Soundness**: {φ: KB ⊢ φ} ⊆ {φ: KB ⊨ φ} → "nothing but the truth" — 유도한 것은 모두 참
- **Completeness**: {φ: KB ⊢ φ} ⊇ {φ: KB ⊨ φ} → "the whole truth" — 참인 것은 모두 유도

★ 필수암기 Soundness = {⊢} ⊆ {⊨} (유도 ⊂ 귀결) · Completeness = {⊢} ⊇ {⊨} (유도 ⊃ 귀결). Modus ponens alone: sound ✓, complete ✗

Soundness 검증 — Modus Ponens

- 질문: Rain ∩ Rain→Wet 가 Wet 을 entail하는가?
- → Entailed, thus sound!
- 반례: Wet ∩ Rain→Wet 로 Rain 을 유도하는 규칙 → Not entailed → unsound

Completeness 반례 — Modus Ponens 불완전

- KB = {Rain, Rain∨Snow→Wet}, φ = Wet
- 의미적: KB ⊨ Wet (Rain이 true이므로 Rain∨Snow true → Wet true)
- 구문적: KB ⊢/Wet (Modus ponens로 도달 불가) → Incomplete!

	Modus Ponens	Resolution
Formula 제한	Horn clauses만	임의 clause
Completeness	완전 (Horn 한정)	완전 (일반)
시간복잡도	선형 O(n)	지수 O(exp)
표현력	덜 표현적	더 표현적

추론 규칙 트레이드오프 (p.23, 34)

Horn Clauses & Modus Ponens 완전성 p.18-22

Q. Horn clause의 형태는?

Q. modus ponens 규칙은?

Q. Horn에서만 완전한 이유·복잡도?

Definite Clause 정의 — $(p_1 \wedge \cdots \wedge p_n) \rightarrow q$ 형태. $p_1,\cdots,p_n,q$ 는 **명제 기호**.

- 예: (Rain ∧ Snow) → Traffic ✓ · Traffic ✓ (body 없음)
- 비예: Rain ∧ Snow ✗ · ¬Traffic ✗ · (Rain ∧ Snow) → (Traffic ∨ Peaceful) ✗
- ⟹ *head는 단일 양의 기호*, body는 conjunct만 허용

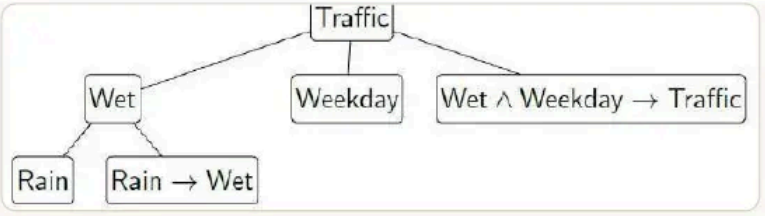
Horn Clause 정의 — Definite clause 또는 goal clause ($p_1 \wedge \cdots \wedge p_n \rightarrow$ false) 의 합집합.

- Definite: (Rain ∧ Snow) → Traffic
- Goal: Traffic ∧ Accident → false [≡ ¬(Traffic ∧ Accident)]
- 이접(disjunction) 표현: 각 Horn clause = **음수 literal 여럿 + 양수 literal 최대 1개**의 절(clause)

Modus Ponens (일반형)

- $p_1,\cdots,p_n$, $(p_1 \wedge \cdots \wedge p_n) \rightarrow q \vdash q$
- 예: Wet, Weekday, Wet∧Weekday→Traffic ⊢ Traffic
- 이접 재작성: p, ¬p∨q ⊢ q — p와 ¬p가 **상쇄(cancel)**

✎ 핵심 정리: Modus ponens on Horn clauses = **완전(complete)**. KB가 Horn clause만 포함하고 φ가 entailed 명제 기호이면, modus ponens가 반드시 φ를 도출. 즉 KB ⊨ φ ⟺ KB ⊢ φ (Horn KB에서만).



Modus ponens derivation tree: 잎=원래 KB 공식, 내부=modus ponens 결과 (p.22)

Resolution 추론 규칙

p.25-32 · 일반 clause · 지수 시간

Q. resolution 규칙은?

Q. CNF 변환 5단계는?

Q. refutation(¬φ 추가)의 원리는?

Literal & Clause

- **Literal**: 명제 기호 p 또는 그 부정 ¬p
- **Clause**: literal의 이접(disjunction) — ¬A∨B∨¬C∨D∨¬E∨F
- **Horn clause**: 양의 literal 최대 1개인 clause

Resolution Rule 정의 — 두 clause 에서 상호 보완 literal (ℓ 과 $\neg\ell$)을 제거해 새 clause를 생성.

- $(\ell_1 \vee \cdots \vee \ell_k \vee \ell),\ (\neg\ell \vee m_1 \vee \cdots \vee m_n) \vdash \ell_1 \vee \cdots \vee \ell_k \vee m_1 \vee \cdots \vee m_n$
- 예: Rain∨Snow, ¬Snow∨Traffic ⊢ Rain∨Traffic
- **Modus ponens**: resolution의 특수 케이스 — p, ¬p∨q ⊢ q
- Soundness: Rain∨Snow ∩ ¬Snow∨Traffic ⊆ (Rain∨Traffic) → Entailed, sound

> ★ 필수암기 Resolution = 완전(complete) + 건전(sound) for 명제논리. 단, 시간복잡도 {지수} — clause가 많은 literal 포함 가능 → 지수적 확장.

CNF (Conjunctive Normal Form) — clause들의 conjunction(∧). Resolution은 CNF에서만 직접 작동.

- 예: (A∨B∨¬C) ∧ (¬A∨B)
- 정리: 모든 명제논리 공식 φ는 동등한 CNF φ' 로 변환 가능 — 표현력 손실 없음

CNF 변환 알고리즘 (5단계)

```
1. Eliminate ↔ (biconditional):
   φ↔ψ  →  (φ→ψ)∧(ψ→φ)
2. Eliminate → (implication):
   φ→ψ  →  ¬φ∨ψ
3. Move ¬ inwards (de Morgan):
   ¬(φ∧ψ)  →  ¬φ∨¬ψ
   ¬(φ∨ψ)  →  ¬φ∧¬ψ
4. Eliminate double negation:
   ¬¬φ  →  φ
5. Distribute ∨ over ∧:
   φ∨(ψ∧χ)  →  (φ∨ψ)∧(φ∨χ)
```

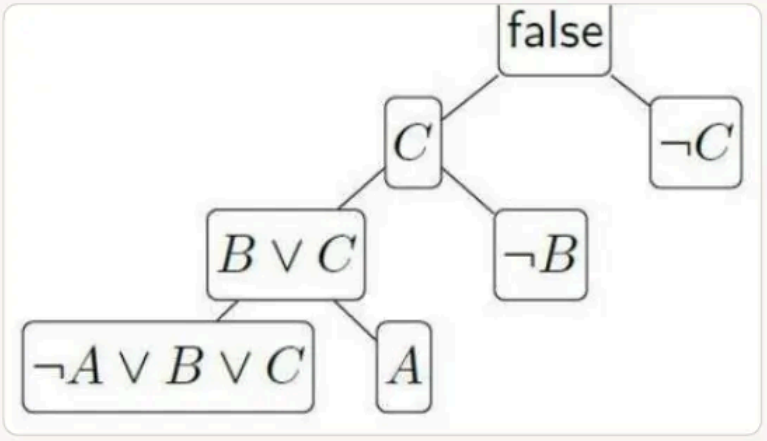
예: (Summer→Snow)→Bizzare → (Summer∨Bizzare)∧(¬Snow∨Bizzare)

Refutation (반박) 기반 Resolution — 핵심 관계: KB ⊨ φ ⟺ KB ∪ {¬φ} 가 unsatisfiable

Resolution-Based Inference

```
Input: KB, φ
1. Add ¬φ to KB
2. Convert all formulas to CNF
3. repeat:
     apply resolution rule to pairs of clauses
     add new clause to KB
4. return ENTAILED if derive false (empty clause)
   return UNKNOWN otherwise
```

false(빈 clause) 도출 = KB∪{¬φ}이 비일관(unsatisfiable) → KB⊨φ 증명됨



Resolution refutation tree 예시: false 도출 → KB entails C (p.32)

Resolution 예제 추적 — KB = {A→(B∨C), A, ¬B}, KB ⊨ C?

- Step 1: Add ¬C → KB' = {A→(B∨C), A, ¬B, ¬C}
- Step 2: CNF → KB' = {¬A∨B∨C, A, ¬B, ¬C}
- Step 3: resolve ¬A∨B∨C + A → B∨C
- Step 4: resolve B∨C + ¬B → C
- Step 5: resolve C + ¬C → false (빈 clause) → KB ⊨ C 증명

복잡도 & 최종 비교 p.33-34

Q. modus ponens vs resolution 비교?

Q. 완전성·복잡도 트레이드오프?

시간복잡도

- Modus ponens: 매 적용마다 단일 명제 기호 1개 추가 → 최대 n번 → O(n) 선형
- Resolution: 매 적용마다 많은 literal 포함 clause 추가 → O(exp) 지수

	Horn+Modus Ponens	Any+Resolution
Formula	Horn clauses만	임의 clause
완전성	완전	완전
시간	선형 O(n)	지수 O(exp)
표현력	덜 표현적	더 표현적

Horn+MP vs Resolution 트레이드오프

⚠ 함정 Modus ponens가 incomplete한 이유: Horn clause 형식을 벗어난 공식(예: A∨B→C)은 전제로 쓸 수 없음 → 참인 결론을 도달 못할 수 있음. 해결책 ①: Horn clauses로 제한(표현력↓) / ②: Resolution 사용(시간↑).

★ 필수암기 Modus ponens: sound+Horn완전·선형 · Resolution: sound+일반완전·지수 · CNF변환 5단계(↔ → ¬안으로 ¬¬제거 ∨/∧분배) · refutation: KB⊨φ ⟺ KB∪{¬φ} unsatisfiable

요약 **Soundness** $\{\vdash\} \subseteq \{\models\}$ · **Completeness** $\{\vdash\} \supseteq \{\models\}$ · Modus ponens는 **Horn clause**에서만 완전(선형) · Resolution은 임의 clause 완전하나 지수 · CNF 5단계 + refutation($KB \cup \{\neg\varphi\}$ unsat $\iff KB \models \varphi$).

← 인공지능 · 포스텍 목차로